WS-CON-001

Workstream Contribution Record and Compensation Boundary

v0.1 Specification

The definitive v0.1 contract for contribution recognition, frozen compensation terms, project-scoped awards, exact adapter fulfillment, centralized authorization, and deterministic artifact-backed contribution evidence.

STATUS	Locked for implementation handoff
MATURITY	Design complete; not yet runtime-proven
BOUNDARY	Authorization / Contribution / Compensation / Evidence
DATE	15 July 2026

Contents

This document defines the complete Workstream v0.1 contribution and compensation boundary, including its Authorization Service, external fulfillment adapter, and Artifact Storage integration contracts. Section numbering follows the locked Markdown source.

1. Purpose

This specification defines how Workstream recognizes completed contributor work after a valid human review decision.

It establishes:

- when a submitter has produced a recognized contribution;
- when a reviewer has produced a recognized contribution;
- which exact Authorization Service permissions and resource contexts protect contribution, policy, award, adapter-binding, reconciliation, and callback operations;
- how contribution recognition remains independent of the review outcome;
- how a global contribution history remains filterable by project;
- how project compensation terms are versioned and frozen before work begins;
- how money and project-points awards are derived deterministically;
- how Workstream hands fulfillment to external adapters;
- how adapter acknowledgements differ from fulfillment results;
- how Workstream records fulfillment results without mutating contribution history;
- how Workstream creates a deterministic immutable evidence bundle for each contribution without duplicating artifact bytes or making Artifact Storage canonical;
- how authorized readers retrieve contribution evidence through Workstream ArtifactBinding references;
- how duplicate events, callbacks, retries, failures, and races are handled;
- which responsibilities stop at the Workstream boundary.

The fundamental model is:

```
Valid Review recorded
|
+--> Reviewer ContributionRecord
| +--> ContributionRecorded
| +--> zero or more CompensationAwards
|
+--> if decision = accept:
Submitter ContributionRecord
+--> ContributionRecorded
+--> zero or more CompensationAwards

Each CompensationAward
|
+--> CompensationFulfillmentRequested
|
+--> external money or project-points adapter
|
+--> CompensationFulfillmentReported
|
+--> immutable fulfillment receipt
+--> mutable status projection
```

Workstream records the fact that work was performed and the exact project-scoped compensation authorized for it. Workstream does not implement payment-provider transactions, point balances, or provider ledgers.

2. Implementation Status and Proof Boundary

The coding agent MUST preserve the difference between proven behaviour, locked design, and deferred work.

Lifecycle area	Status
Project guide, submission policy, task lock, submission finalization, and durable checker run	Proven
Transition through review_pending	Proven
Reviewer routing, leases, immutable decisions, and revision chain	Locked by WS-REV-001; not yet runtime-proven
ActorProfile and centralized Authorization Service contract	Locked by WS-AUTH-001; implementation proof required before protected WS-CON APIs
Reviewer Contribution Record for every completed Review	Locked by this specification
Submitter Contribution Record after accept	Locked by this specification
Project CompensationPolicy and immutable versions	Locked by this specification
Money and project-points CompensationAwards	Locked by this specification
External fulfillment event and callback boundary	Locked by this specification
Contribution evidence-bundle projection and retention	Locked by this revision; not yet runtime-proven
Provider payment requests, attempts, settlement, balances, and reconciliation	External and deferred
Project-points ledger and balances	External and deferred
Reputation scoring formula and aggregate	Deferred
Adjudication and contribution adjustment	Deferred

2.1 Starting preconditions

This specification begins when WS-REV-001 is recording a valid Review against an active ReviewLease.

The Review decision may be:

- accept;
- needs_revision;
- reject.

No Contribution Record may be created for:

- an automated checker result;
- a pending or merely claimed review;
- an expired lease;
- a manually released lease;
- an administratively revoked lease;

- an invalid or rolled-back Review transaction.

2.2 Narrow precedence over WS-REV-001

WS-REV-001 section 12.1 uses ContributionRecordRequested as a temporary integration event because contribution creation was deferred to the next specification.

After WS-CON-001 is implemented, the following rules supersede only that temporary seam:

1. Every valid Review transaction creates the reviewer Contribution Record directly.
2. An accept Review transaction additionally creates the submitter Contribution Record directly.
3. Those records, their CompensationAwards, and their outbox events are committed atomically with the Review.
4. ContributionRecordRequested MUST NOT be emitted by the completed implementation.
5. ContributionRecorded and CompensationFulfillmentRequested are the canonical downstream events.

All other WS-REV-001 authority, routing, lease, revision, reject, and immutability rules remain unchanged.

For authorization identifiers only, `WS-AUTH-001-A` has precedence over any preliminary broad names still printed in an earlier WS-REV-001 revision. The combined implementation uses `review.decision.record`, `artifact.recovery.request`, `artifact.recovery.execute`, `outbox.dispatch`, and the other locked granular catalogue identifiers without aliases.

---

3. Normative Language

The terms MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, and MAY are normative.

- MUST or REQUIRED means the implementation is non-conformant if the rule is absent.
- MUST NOT means the behaviour is prohibited.
- SHOULD means deviation requires an explicit Architecture Decision Record.
- MAY means the implementation choice is allowed without changing domain behaviour.

Database time is authoritative for all transaction timestamps. Decimal quantities MUST never be represented or calculated using binary floating-point types.

Version notation: Workstream and this lifecycle specification are release v0.1. The `/v1` API prefix, `event_version: 1`, event idempotency suffix `:v1`, route-key suffix `-v1`, and references such as `SubmissionVersion(v1)` are independent API, event-schema, route, or submission-version identifiers. None denotes a Workstream v1.0 release.

---

4. Locked v0.1 Decisions

The following decisions are not left to coding-agent interpretation.

1. Contribution Records are globally addressable in Workstream and always carry a mandatory project_id.
2. Global contribution history can be filtered by project, contributor, task, contribution type, and time.

3. A valid completed Review always creates exactly one reviewer Contribution Record, regardless of accept, needs_revision, or reject.
4. An accept Review additionally creates exactly one submitter Contribution Record.
5. Needs-revision and reject Reviews do not create submitter Contribution Records.
6. Every completed revision-review round is an independently recognized reviewer contribution.
7. Expired, released, and revoked review leases create no contribution and no compensation.
8. Contribution Records are immutable and have no mutable fulfillment fields.
9. Review, contribution, award, and outbox creation occur in one database transaction.
10. Every project MUST bind an active published CompensationPolicyVersion before new TaskAssignments or ReviewLeases can be created.
11. Unpaid work uses an explicit policy rule with no awards; missing policy configuration is never treated as unpaid.
12. V0.1 compensation is fixed per project and contribution type.
13. V0.1 has no contributor-specific rates, manually negotiated rates, compensation tiers, or task-specific overrides.
14. Submitter compensation terms are frozen on TaskAssignment creation.
15. Reviewer compensation terms are frozen on successful ReviewLease creation.
16. A later project-policy change never alters an existing TaskAssignment, ReviewLease, Contribution Record, or CompensationAward.
17. V0.1 supports only money and project_points compensation instruments.
18. Monetary awards are project obligations even when their unit is a standard currency.
19. Project points belong to a project-specific namespace and are not global Workstream points.
20. A contribution may create no award, one award, or two awards: at most one money award and at most one project-points award.
21. Workstream creates immutable CompensationAwards and emits fulfillment events.
22. External adapters own payment requests, provider attempts, finance approval, point ledgers, balances, provider retries, and provider reconciliation.
23. Workstream retries event delivery only until the external adapter acknowledges receipt.
24. Adapter delivery acknowledgement is not proof of compensation fulfillment.
25. Fulfillment is reported later through an authenticated, idempotent callback.
26. V0.1 fulfillment outcomes are fulfilled or failed; partial fulfillment is prohibited.
27. A failed award may later receive a valid fulfilled receipt.
28. A fulfilled award is terminal and cannot later become failed.
29. Fulfillment responses create immutable receipts and update a derived status projection; they never mutate the Contribution Record or CompensationAward.
30. Reputation consumes ContributionRecorded independently of compensation fulfillment.
31. Reputation scoring formulas and reputation aggregates are not implemented by this specification.
32. Acceptance is final in v0.1; no adjudication or adjustment lifecycle exists.
33. All provider-specific and ledger-specific objects remain outside Workstream.
34. `WS-AUTH-001` is the sole authority for ActorProfile resolution, AdminRoleGrant evaluation, contributor self-read authority, project scope, service-actor status, and authorization decisions.

35. Finance Authority is the only human Admin role that creates, edits, publishes, retires, or activates compensation policy and manages compensation adapter bindings in v0.1.
36. Operator may execute authorized compensation delivery/reconciliation operations but cannot publish policy, alter awards, or mark fulfillment complete.
37. An external fulfillment adapter reports results only through an active service ActorProfile and the exact frozen ProjectCompensationAdapterBinding for the referenced award.
38. Workstream owns ContributionRecord, CompensationAward, fulfillment receipt, projection, and lifecycle meaning. Artifact Storage owns contribution-evidence bundle bytes, content identity, manifests, provenance, retention, and optional semantic indexing.
39. Every ContributionRecord produces one asynchronous, deterministic contribution-evidence projection request after the canonical Review/contribution transaction commits.
40. Artifact projection failure never rolls back or mutates a committed Review, ContributionRecord, CompensationAward, or fulfillment receipt.
41. Human Identity Issuer tokens are never forwarded to Artifact Storage or compensation adapters. Workstream uses separately authorized service identities and least-privilege scopes.
42. Public contribution evidence references use Workstream ArtifactBinding IDs. Raw provider CIDs, arbitrary URLs, filesystem paths, credentials, and provider tokens are not public authority.

5. Actor and Authority Model

5.1 Relevant actors

```
Workstream Actor
├── Admin
│ ├── Access Administrator
│ ├── Operator
│ ├── Project Manager
│ ├── Finance Authority
│ └── Audit Authority
│
├── Contributor
│ ├── Submitter
│ ├── Reviewer
│ └── Both
│
└── External Service Actor
├── Money Fulfillment Adapter
└── Project Points Fulfillment Adapter
```

External adapters are service actors. They are not Admins or Contributors and receive no authority over Reviews, tasks, assignments, policies, or Contribution Records.

These are authority domains, not separate human profiles. One Identity Issuer subject resolves to one Workstream ActorProfile. An ActorProfile may hold AdminRoleGrants and project contributor grants independently; neither implies the other.

5.2 Normative relationship to WS-AUTH-001

`WS-AUTH-001` is authoritative for:

- Identity Issuer token verification;

- ActorProfile and ActorIdentityLink provisioning and state;
- AdminRoleGrant and ProjectRoleGrant evaluation;
- registered permission identifiers;
- system, project and record scope;
- service ActorProfile provisioning;
- authorization decisions, revocation and audit linkage.

`WS-AUTH-001-A` is authoritative where it replaces broad preliminary permission names with the closed granular action/resource catalogue. This document consumes those exact identifiers and introduces no compatibility alias.

This specification is authoritative for:

- compensation policy and adapter-binding lifecycle guards;
- contribution and award creation invariants;
- frozen compensation terms;
- adapter-binding ownership of fulfillment callbacks;
- immutable receipt and projection behavior;
- contribution-evidence ArtifactBinding and projection guards.

Every protected request first obtains an AuthorizationDecision. An allowed decision permits an attempt; WS-CON domain services still enforce policy state, frozen references, ownership, adapter binding, award state, artifact visibility and transaction invariants.

5.3 Identity boundary

All human and service requests pass through the Identity Issuer boundary.

```
Identity Issuer token
-> TokenVerifier verifies issuer, subject, audience, expiry and coarse scopes
-> AuthorizationService resolves current ActorProfile and identity link
-> Workstream loads the canonical policy, contribution, award, binding or receipt resource
chain
-> AuthorizationService evaluates the registered permission and canonical scope
-> WS-CON service applies compensation, ownership and state guards
-> operation proceeds only when every layer allows it
```

A valid service token is insufficient by itself. The service actor must also match the adapter binding frozen on the referenced CompensationAward.

The human or service token is consumed at the Workstream boundary. It is never forwarded to Artifact Storage or from Workstream to an external fulfillment provider.

5.4 Required permission mapping

Operation	Required WS-AUTH permission	Additional WS-CON guard
Read own contributions and composed compensation status	`contribution.read_self`	ContributionRecord.contributor_id equals ActorProfile.id
Read project contribution history	`contribution.read_project`	Covered project scope
Read compensation policy	`compensation.policy.read`	Covered project scope and disclosure projection

Operation	Required WS-AUTH permission	Additional WS-CON guard
Read compensation award/receipt detail	`compensation.award.read`	Beneficiary subject guard or covered project scope and disclosure policy
Create compensation-policy draft	`compensation.policy.create_draft`	Finance Authority; canonical Project target
Replace compensation-policy draft	`compensation.policy.update_draft`	Finance Authority; canonical draft-version target
Publish and activate compensation policy	`compensation.policy.publish`	Finance Authority; current-version and policy-state guards
Retire compensation policy	`compensation.policy.retire`	Finance Authority; current-version and replacement guards
Read adapter binding	`compensation.adapter_binding.read`	Covered project scope; no credentials
Create adapter binding	`compensation.adapter_binding.create`	Finance Authority; active registered service actor and instrument compatibility
Suspend adapter binding	`compensation.adapter_binding.suspend`	Finance Authority; binding active and canonical project scope
Resume adapter binding	`compensation.adapter_binding.resume`	Finance Authority; binding suspended and canonical project scope
Retire adapter binding	`compensation.adapter_binding.retire`	Finance Authority; dependency guards pass
Request compensation delivery reconciliation	`compensation.delivery.reconcile`	Finance Authority for covered project work; system-scoped Operator only for operational delivery incident recovery; neither may change award/receipt truth
Read operational status	`operations.status.read`	Covered operational scope
Rebuild compensation/evidence projection	`operations.projection.rebuild`	Provisioned system actor or authorized Operator
Request recovery of uncertain contribution-evidence artifact operation	`artifact.recovery.request`	Covered Operator; worker owns execution lease
Execute artifact recovery	`artifact.recovery.execute`	Fixed recovery system actor; current execution generation
Create contribution-evidence ArtifactBinding	`artifact.binding.create`	Fixed projection system actor; existing ContributionRecord target; verified receipt and exact owner/project/role
Retrieve contribution evidence internally	`artifact.retrieve`	Fixed retrieval system actor after caller's contribution-read decision; verified binding and disclosure guard
Report fulfillment callback	`compensation.fulfillment.report`	Active service ActorProfile equals frozen binding adapter_actor_id
Materialize ContributionRecord	`contribution.materialize`	Fixed contribution system actor; canonical Review and uniqueness
Materialize CompensationAward	`compensation.award.materialize`	Fixed compensation system actor; frozen policy and uniqueness
Dispatch compensation delivery	`compensation.delivery.dispatch`	Fixed dispatcher system actor; frozen route and idempotency
Dispatch outbox event	`outbox.dispatch`	Fixed dispatcher system actor; current delivery generation
Read/export compensation audit chain	`audit.read` / `audit.export`	Covered Audit or other explicitly authorized scope

`compensation.fulfillment.report` is the locked service permission from `WS-AUTH-001-A`. It is supplied only through the exact ProjectCompensationAdapterBinding frozen on an existing award. The binding must be active, or suspended only for completion of already-issued work; the permission is never granted through a human AdminRoleGrant or ProjectRoleGrant. A retired binding grants no new mutation authority; the exact bound service actor may receive the already-stored response only for a byte-equivalent idempotent replay of a previously accepted callback.

`artifact.recovery.request` is shared with the updated `WS-REV-001` Artifact Storage integration. It authorizes an Operator to request recovery; a fixed system worker executes and fences the same recovery attempt through `artifact.recovery.execute`.

`WS-AUTH-001-A` currently maps `artifact.binding.create` to task/submission evidence. Before WS-CON implementation, its resource mapping MUST be additively extended to permit a fixed Workstream projection system actor to target an existing ContributionRecord for artifact_role `contribution_evidence_bundle`. The permission string is reused; no alias or new human authority is introduced. The public evidence route keeps `contribution.read_self` or `contribution.read_project` as its primary caller permission; the internal retrieval actor separately uses `artifact.retrieve` after the caller decision succeeds.

5.5 Human authority matrix

Action	Authorized human authority in v0.1
Manage any compensation-policy lifecycle stage	Finance Authority within covered project scope
Manage money or project-points adapter binding	Finance Authority within covered project scope
Read own Contribution Records	Contributor owning the records
Read project Contribution Records	Project Manager, Finance Authority, Audit Authority, or operationally authorized Operator within covered scope
Read global cross-project contribution history	Contributor reading only their own history; Admin cross-project analysis uses separately authorized audit/operational projections, not a contribution-history bypass
Read monetary award and fulfillment detail	Finance Authority, Audit Authority, or operationally authorized Operator within covered scope
Execute project compensation reconciliation	Finance Authority within covered project scope
Request operational compensation delivery incident recovery	System-scoped Operator with `compensation.delivery.reconcile`; fixed dispatcher/recovery actor executes
Request eligible artifact recovery	Covered Operator with `artifact.recovery.request`; fixed recovery actor executes
Create ContributionRecord or CompensationAward	No human authority; system transaction only
Report fulfillment	No human authority; bound service actor only

Project Manager may read project contribution history needed for project management but cannot create, publish, activate, retire or edit compensation policy and cannot manage adapter bindings solely by that role. Operator recovers delivery/runtime state but cannot change economic policy, award quantity, or fulfillment truth.

5.6 Canonical resource contexts

The authorization service receives canonical relationships loaded from Workstream records, never trusted from request paths or callback payload fields.

Resource	Canonical project resolution	Required guard attributes
CompensationPolicy/Version	stored project_id, cross-checked through owning policy and Project	status; version; current active reference; instruments
AdapterBinding	stored project_id	instrument; adapter_actor_id; status; reference usage
ContributionRecord	source Review -> SubmissionVersion -> Task -> Project	contributor owner; type; source review; immutable state
CompensationAward	ContributionRecord -> Project and frozen policy/binding	contributor; instrument; binding; quantity; fulfilled state

Resource	Canonical project resolution	Required guard attributes
FulfillmentReceipt	Award -> ContributionRecord -> Project	adapter binding; external event; current fulfilled receipt
Contribution ArtifactBinding	Binding owner -> ContributionRecord -> source Review/Task -> Project	artifact role; visibility; integrity/projection state

Any mismatch fails closed with `resource_project_mismatch` and produces an integrity alert.

5.7 Authorization evaluation points

Authorization MUST be evaluated:

1. before returning contribution, award, policy, binding, receipt or evidence metadata;
2. inside policy publication/activation/retirement transactions;
3. inside adapter-binding mutation transactions;
4. inside delivery reconciliation mutations;
5. inside every fulfillment callback transaction after loading the award and binding;
6. again before any actor-attributed deferred job commits;
7. before every contribution-evidence retrieval.

Routers MUST NOT query AdminRoleGrant tables, inspect role strings, infer project scope from request JSON, or reproduce callback-binding authorization.

Every route and asynchronous command declares exactly one primary permission from `WS-AUTH-001-A`. Derived writes in the Review transaction additionally execute through the fixed `contribution.materialize` and `compensation.award.materialize` system authorities while retaining the originating `review.decision.record` decision as causation.

5.8 System-owned creation

ContributionRecord, CompensationAward, their canonical outbox events and evidence-projection requests are created inside the authorized Review decision transaction. The system actor records the derived action, while the originating Review authorization decision remains linked as causation.

Background dispatch, projection and reconciliation workers use provisioned service ActorProfiles with explicit system permissions. They do not fabricate a Finance Authority, Operator or Contributor identity.

No API permits an Admin, Contributor, or adapter to create or edit a Contribution Record directly.

---

6. Core Object Model

6.1 ContributionRecord

ContributionRecord is the permanent recognition that one contributor completed one eligible unit of work.

```
ContributionRecord:
id: uuid
project_id: uuid
task_id: uuid
contributor_id: uuid

contribution_type: enum [
accepted_submission,
completed_review
]

source_review_id: uuid
source_review_lease_id: uuid | null
source_task_assignment_id: uuid
submission_version_id: uuid

compensation_policy_version_id: uuid

source_submission_artifact_digest: string
recorded_at: timestamp
recorded_by: system_actor_id

correlation_id: string
causation_event_id: uuid | null
```

ContributionRecord invariants

- The record is immutable from insertion.
- project_id, task_id, contributor_id, source Review, assignment, submission version, and compensation version MUST all belong to one internally consistent chain.
- contribution_type = accepted_submission requires Review.decision = accept.
- contribution_type = accepted_submission requires contributor_id to equal the submission creator and TaskAssignment contributor.
- contribution_type = completed_review requires contributor_id to equal Review.reviewer_id.
- contribution_type = completed_review requires source_review_lease_id to equal Review.review_lease_id.
- accepted_submission records reference the active submitter TaskAssignment completed by the accept Review.
- completed_review records reference the TaskAssignment whose work was reviewed but do not attribute that assignment to the reviewer.
- source_submission_artifact_digest equals the immutable digest locked to the reviewed SubmissionVersion. For `completed_review`, it identifies the work judged; it is not misrepresented as the reviewer's own artifact.
- compensation_policy_version_id comes from the frozen TaskAssignment field for accepted_submission.
- compensation_policy_version_id comes from the frozen ReviewLease field for completed_review.
- Exactly one accepted_submission record may exist per accepted Review.
- Exactly one completed_review record may exist per Review.
- An accept Review therefore creates exactly two Contribution Records unless one of the contributors is represented by a prohibited self-review, which WS-REV-001 prevents.
- Needs-revision and reject Reviews create exactly one Contribution Record: completed_review.
- A record may exist even when its compensation policy produces no awards.
- The record has no payment_status, points_status, mutable status, deleted_at, or editable metadata.

Recommended uniqueness constraints:

```
UNIQUE(source_review_id, contribution_type)
UNIQUE(id, project_id)
```

6.2 CompensationPolicy

CompensationPolicy is the stable project-owned policy identity.

```
CompensationPolicy:
id: uuid
project_id: uuid
name: string
status: enum [draft, active, retired]
current_published_version_id: uuid | null

created_by: actor_id
created_at: timestamp
retired_by: actor_id | null
retired_at: timestamp | null
```

One project may have multiple historical policy identities, but at most one CompensationPolicy and, when the project is configured for new work, exactly one published version are active at a time. A newly created policy remains draft with a null current_published_version_id until its first version is published.

6.3 CompensationPolicyVersion

```
CompensationPolicyVersion:
id: uuid
compensation_policy_id: uuid
project_id: uuid
version_number: int
status: enum [draft, published, retired]

created_by: actor_id
created_at: timestamp
published_by: actor_id | null
published_at: timestamp | null
retired_by: actor_id | null
retired_at: timestamp | null
```

Version invariants

- version_number starts at 1 and increases monotonically within one CompensationPolicy.
- Draft versions may be edited only through explicit draft APIs.
- Publishing validates the complete version and makes every field and award definition immutable.
- A published version may be retired but never edited or deleted.
- Retiring a version does not invalidate frozen references from assignments or leases.
- A retired version remains valid for contribution and award creation when it was frozen before retirement.
- At most one published version is active for new work within a project.

6.4 CompensationRule

Every policy version contains exactly one rule for each contribution type. This makes unpaid work explicit rather than indistinguishable from missing configuration.

```
CompensationRule:
id: uuid
compensation_policy_version_id: uuid
project_id: uuid
contribution_type: enum [
accepted_submission,
completed_review
]
compensation_mode: enum [compensated, unpaid]
```

Rule invariants

- Exactly one accepted_submission rule and one completed_review rule exist in every publishable version.
- A compensated rule has at least one and at most two award definitions.
- An unpaid rule has zero award definitions.
- Rule and definitions become immutable together when the version is published.
- V0.1 rules have no condition expression, priority, fallback, multiplier, or outcome selector.

6.5 CompensationAwardDefinition

Each published version defines fixed awards for the two contribution types.

```
CompensationAwardDefinition:
id: uuid
compensation_rule_id: uuid
compensation_policy_version_id: uuid
project_id: uuid

contribution_type: enum [
accepted_submission,
completed_review
]

instrument_type: enum [
money,
project_points
]

unit_code: string
quantity: decimal
adapter_binding_id: uuid
```

Definition invariants

- At most one definition exists for each contribution_type and instrument_type within a version.
- quantity MUST be greater than zero.
- Money unit_code MUST be an uppercase ISO 4217 currency code configured for the project.
- Project-points unit_code MUST match the project-scoped point unit configured by the policy.
- A points unit has identity (project_id, unit_code). The same text in another project is a different unit.
- adapter_binding_id must match the same project and instrument.
- Published definitions are immutable.

- An explicitly unpaid rule has zero award definitions.
- Zero-quantity award definitions are prohibited.
- V0.1 rule evaluation is exact matching on contribution_type only. No other condition, multiplier, tier, score, task class, contributor class, or outcome rule is allowed.

6.6 Frozen compensation references

WS-CON-001 extends Project and two existing work objects.

```
Project:
# existing fields omitted
active_compensation_policy_version_id: uuid | null
```

```
TaskAssignment:
# existing fields omitted
submitter_compensation_policy_version_id: uuid
```

```
ReviewLease:
# existing fields omitted
reviewer_compensation_policy_version_id: uuid
```

The references are set once:

- TaskAssignment freezes the project active version in the assignment-creation transaction.
- ReviewLease freezes the project active version in the successful claim transaction.
- Neither field may be updated later.
- Policy retirement does not change either field.

6.7 CompensationAward

CompensationAward is Workstream's immutable statement of what the project authorized for one contribution.

```
CompensationAward:
id: uuid
project_id: uuid
contribution_record_id: uuid
contributor_id: uuid

compensation_policy_version_id: uuid
award_definition_id: uuid
adapter_binding_id: uuid

instrument_type: enum [
money,
project_points
]

unit_code: string
quantity: decimal

created_at: timestamp
correlation_id: string
```

CompensationAward invariants

- Immutable from insertion.
- Project, contributor, policy version, and contribution must match.
- Instrument, unit, quantity, and adapter binding are copied exactly from the published definition.
- At most one award exists per contribution and instrument type.
- A completed_review award does not depend on Review.decision.
- No award may be created for a released, expired, or revoked lease.
- No award may be created by an adapter callback.
- Failed or fulfilled status is never stored on this record.

Recommended uniqueness constraint:

```
UNIQUE(contribution_record_id, instrument_type)
```

6.8 ProjectCompensationAdapterBinding

The binding identifies the logical adapter authorized for one project and instrument.

```
ProjectCompensationAdapterBinding:
id: uuid
project_id: uuid
instrument_type: enum [money, project_points]
adapter_actor_id: actor_id
route_key: string
status: enum [active, suspended, retired]

created_by: actor_id
created_at: timestamp
suspended_at: timestamp | null
retired_at: timestamp | null
```

Binding invariants

- At most one active binding exists per project and instrument.
- Published award definitions reference a binding for the same project and instrument.
- The binding identity is frozen into each CompensationAward.
- Endpoint, credentials, and provider configuration are deployment secrets and are not domain fields.
- Suspending a binding pauses new delivery but does not cancel or alter awards.
- Retiring a binding is prohibited while it is referenced by an active policy version, by unfinished work whose frozen policy version contains it, or by an unfulfilled award.
- A service actor bound to money cannot report project-points fulfillment and vice versa.

6.9 CompensationFulfillmentReceipt

Each valid callback creates an immutable receipt.

```
CompensationFulfillmentReceipt:
id: uuid
compensation_award_id: uuid
project_id: uuid
adapter_binding_id: uuid

external_event_id: string
reported_status: enum [fulfilled, failed]

external_reference: string | null
fulfilled_quantity: decimal | null
fulfilled_at: timestamp | null

failure_code: string | null
failure_message: string | null

reported_at: timestamp
received_at: timestamp
correlation_id: string
```

Receipt invariants

- Immutable from insertion.
- external_event_id is globally unique per adapter binding.
- fulfilled requires external_reference, fulfilled_quantity, and fulfilled_at.
- fulfilled_quantity must equal the CompensationAward quantity exactly.
- failed requires failure_code.
- failed must have null fulfilled_quantity and fulfilled_at.
- Project, binding, contributor, instrument, unit, and authorized quantity cannot be changed by the callback.
- Multiple failed receipts may exist for one award when their external_event_id values differ.
- At most one fulfilled receipt may exist for one award.
- A fulfilled award rejects all later failed or fulfilled callbacks except an exact idempotent replay of the accepted fulfilled callback.

6.10 CompensationStatusProjection

This projection exists only for efficient reads and operations.

```
CompensationStatusProjection:
compensation_award_id: uuid
delivery_status: enum [
pending_delivery,
acknowledged_by_adapter
]
fulfillment_status: enum [
pending,
failed,
fulfilled
]

latest_receipt_id: uuid | null
external_reference: string | null
last_failure_code: string | null
delivered_at: timestamp | null
fulfilled_at: timestamp | null
updated_at: timestamp
```

The projection is mutable and rebuildable. It is not the source of truth. ContributionRecord, CompensationAward, outbox delivery history, and CompensationFulfillmentReceipt are authoritative.

6.11 ContributionEvidenceProjection

Every ContributionRecord receives one rebuildable projection status row for its immutable contribution-evidence bundle.

```
ContributionEvidenceProjection:
contribution_record_id: uuid
project_id: uuid
projection_schema_version: int
status: enum [pending, projected, failed]

artifact_binding_id: uuid | null
content_digest: string | null
provider_receipt_ref: opaque string | null

last_error_code: string | null
last_attempt_at: timestamp | null
projected_at: timestamp | null
updated_at: timestamp
```

The projection row is mutable and rebuildable. ContributionRecord and its source Review chain remain canonical.

Logical evidence-bundle schema

The deterministic bundle is a versioned manifest; it references already retained artifacts rather than duplicating their bytes.

```
ContributionEvidenceBundle:
schema_version: int
contribution_record_id: uuid
contribution_type: accepted_submission | completed_review
project_id: uuid
task_id: uuid
contributor_id: uuid

source_review_id: uuid
source_review_lease_id: uuid | null
source_task_assignment_id: uuid
submission_version_id: uuid
source_submission_artifact_digest: string

locked_task_context_ref: uuid
project_guide_version_ref: uuid
effective_project_policy_version_ref: uuid
review_policy_version_ref: uuid
submission_artifact_binding_ids: list[uuid]
checker_artifact_binding_ids: list[uuid]
review_finding_ids: list[uuid]
review_finding_evidence_binding_ids: list[uuid]
finding_resolution_ids: list[uuid]

compensation_policy_version_id: uuid
compensation_award_ids: list[uuid]
canonical_recorded_at: timestamp
```

For `accepted_submission`, the bundle proves the accepted submission, exact checker/review context, and resulting contribution. For `completed_review`, it proves the exact submission judged, canonical Review, findings/resolutions and evidence used by the reviewer.

Evidence projection invariants

- Exactly one ContributionEvidenceProjection row exists per ContributionRecord.
- `artifact_binding_id`, when present, references a verified generic ArtifactBinding with owner_type `contribution_record` and artifact_role `contribution_evidence_bundle`.
- Bundle project, task, contributor, Review, submission, compensation policy and award references resolve to the same canonical chain.
- Guide, effective-policy and review-policy references are copied from the task's immutable locked context; the worker MUST NOT load the project's current versions as substitutes.
- Bundle generation reads canonical committed Workstream records; it never trusts event payload fields as independent authority.
- The serialized bundle is deterministic for one ContributionRecord and projection schema version.
- Same projection identity and bytes return the same logical storage receipt; changed bytes under the same identity are an integrity conflict.
- Projection failure does not modify or invalidate the ContributionRecord or compensation lifecycle.
- Artifact Storage contains the immutable bundle bytes/manifest; PostgreSQL contains canonical lifecycle truth and projection status/reference.

Canonical serialization contract

- Media type is `application/vnd.workstream.contribution-evidence+json;version=1`.
- JSON is serialized with RFC 8785 JSON Canonicalization Scheme rules and UTF-8 bytes.
- Timestamps use UTC RFC 3339 form with normalized fractional precision.
- UUIDs are lowercase canonical strings.
- All list fields are present, use empty lists rather than null, and are ordered by canonical record creation time followed by id.
- Scalar optional fields use explicit null when the schema permits null.
- Mutable compensation delivery/fulfillment projections, mutable evidence status and provider-local references are excluded from the bundle.
- `content_digest` is lowercase `sha256:<hex>` over the exact canonical bytes.
- A projection schema-version change is required for any semantic or serialization change.

6.12 Artifact Storage ownership boundary

WS-CON reuses the same provider-independent ArtifactStorePort and generic ArtifactBinding contract defined by the updated WS-REV-001 integration. It MUST NOT introduce a compensation-specific byte store or import Local Storage/Flow Node internals.

```
Workstream owns
ContributionRecord and compensation meaning
source Review/task/submission relationships
evidence-bundle schema and projection status
ArtifactBinding meaning and visibility
authorization, audit and outbox state

Artifact Storage owns
serialized immutable bundle bytes
content digest and provider reference
manifest/provenance receipt
retention/pin operation
optional semantic/search projection
```

The required port operations are:

```
class ArtifactStorePort:
async def get_metadata(binding_ref, service_scope): ...
async def retrieve_verified(binding_ref, service_scope): ...
async def verify(binding_ref, expected_digest, service_scope): ...
async def store(stream, source_ref, expected_digest, media_type,
service_scope, idempotency_key): ...
async def ensure_retained(binding_ref, retention_class, owner_ref,
service_scope, idempotency_key): ...
async def store_projection(stream, source_ref, version, expected_digest,
service_scope, idempotency_key): ...
```

The projection identity is:

```
contribution:{contribution_record_id}:evidence-bundle:{projection_schema_version}
```

6.13 Contribution evidence projection flow

The canonical Review/contribution transaction performs no remote storage call. It creates the ContributionRecord, a pending ContributionEvidenceProjection row and a `ContributionEvidenceProjectionRequested` transactional outbox event.

After commit, the projection worker:

1. runs as a provisioned Workstream system actor;
2. loads the ContributionRecord and complete canonical source chain;
3. resolves every referenced Workstream ArtifactBinding and verifies project/task consistency;
4. builds the deterministic versioned bundle;
5. computes the expected digest before the provider call;
6. creates a least-privilege service scope for the project, contribution and projection action;
7. calls ArtifactStorePort.store_projection with the stable operation identity;
8. validates provider receipt source reference, digest, size, media type, scope and operation identity;
9. creates or resolves the verified `contribution_evidence_bundle` ArtifactBinding;
10. applies contribution evidence retention/pinning;
11. updates ContributionEvidenceProjection to `projected`;
12. emits `ContributionEvidenceProjected` audit/integration evidence.

No ContributionRecord, award, receipt or Review is inferred from provider content. Projection always begins from canonical Workstream records.

6.14 Authorized evidence retrieval

Contribution-list endpoints return evidence status and authorized Workstream references only; they do not inline artifact bytes.

When a caller requests contribution evidence:

1. AuthorizationService requires `contribution.read_self` or `contribution.read_project` for the canonical ContributionRecord.
2. Workstream proves the projection ArtifactBinding belongs to that ContributionRecord and project.
3. Workstream creates a least-privilege service retrieval scope.
4. ArtifactStorePort retrieves and verifies the exact digest.
5. Workstream records an attributable access receipt/audit reference.
6. Workstream returns the authorized bundle or approved derivative representation.

Artifact Storage search results are candidate references only. Workstream re-authorizes every result before disclosure.

6.15 Artifact failure semantics

Failure	WS-CON effect
Artifact Storage unavailable after contribution commit	Contribution and awards remain canonical; projection stays pending/failed and retries
Projection receipt digest mismatch	Do not bind receipt; mark failed; raise integrity alert
Same operation identity with different bundle bytes	Integrity conflict; dead-letter for Operator recovery
Evidence bundle unavailable during authorized read	Return `contribution_evidence_unavailable`; do not change contribution or compensation status
Semantic index unavailable	Bundle storage remains valid; retry indexing independently
Retention/pin operation fails	Contribution remains valid; alert and retry retention until policy is satisfied
Provider outcome uncertain	Use the existing idempotent artifact recovery contract; never guess success

An Artifact Storage failure never creates or changes an award, fulfillment receipt, contributor reputation signal, or review outcome.

6.16 Storage-adapter equivalence

The Local Storage adapter used in development/CI and the production Flow Node adapter MUST pass the same contribution-evidence conformance suite.

Provider-specific CID, manifest, pinning, provenance and indexing capabilities may differ. Digest identity, project scope, idempotency, authorization, projection status, retention behavior and Workstream lifecycle effects must remain identical.

The artifact outbox, idempotency and reconciliation rules apply to both adapters. Local Storage may avoid a remote network hop, but a process crash, lost worker acknowledgement, partial filesystem effect or redelivered Celery message can still leave Workstream unable to prove the operation outcome. Recovery is an Artifact Storage orchestration concern, not a Flow Node-specific feature.

7. Project Scope and Global Contribution Views

7.1 Global record identity

ContributionRecord.id is globally unique within Workstream. A contributor's complete history may span projects.

```
Contributor
└── Contribution history
├── Project A
│ ├── accepted_submission
│ └── completed_review
├── Project B
│ └── completed_review
└── Project C
└── accepted_submission
```

7.2 Project-scoped economics

Every CompensationPolicy, policy version, award definition, CompensationAward, adapter binding, and fulfillment receipt has a project_id.

- A Project A policy cannot compensate Project B work.
- Project A points cannot be fulfilled into Project B's point namespace.
- A USD award from Project A remains Project A's obligation; it is not a global Workstream balance.
- Cross-project totals are read-model calculations only.

7.3 Read views

Required views:

- contributor global contribution history;
- contributor history filtered to one project;
- project contribution history;
- task contribution history;
- Contribution Record detail with derived compensation summary;
- Contribution Record detail with contribution-evidence projection status and authorized ArtifactBinding reference;
- project outstanding compensation awards by instrument and age;
- adapter delivery and fulfillment operational view.

8. Compensation Policy Lifecycle

8.1 Create draft

An active Finance Authority whose effective scope covers the project and whose AuthorizationDecision allows `compensation.policy.create_draft` creates a CompensationPolicy and version 1 draft, or creates the next draft version under an existing policy.

Draft creation does not affect existing or new work until publication and activation.

8.2 Validate draft

Publication validation MUST confirm:

- policy and every definition belong to the same project;
- only accepted_submission and completed_review rules exist;
- only money and project_points instruments exist;
- no duplicate instrument exists within a contribution type;
- every quantity is a positive decimal;
- every unit code is valid for its instrument;
- every adapter binding belongs to the project, supports the instrument, and is active;
- both contribution types are explicitly present as either compensated or unpaid;
- no task, contributor, tier, outcome, skill, or reputation condition exists;
- AuthorizationService allows `compensation.policy.update_draft` for draft replacement and `compensation.policy.publish` for publication validation on the canonical resource.

8.3 Publish and activate

Publication and project activation occur in one transaction:

```
validate complete draft
lock project's active compensation policy reference
publish version immutably
set owning CompensationPolicy.status -> active
set owning CompensationPolicy.current_published_version_id -> new version id
retire any different previously active CompensationPolicy identity
set project active_compensation_policy_version_id
retire prior version for new-work selection
write policy audit events
commit
```

The transaction revalidates `compensation.policy.publish` through AuthorizationService after loading and locking the canonical project/version resources. Retirement separately revalidates `compensation.policy.retire`. A router-level role check is insufficient.

Retiring the prior version means it is no longer selected for new assignments or leases. It remains valid for previously frozen work.

If an active version is retired without an immediate replacement, the same transaction sets Project.active_compensation_policy_version_id to null and CompensationPolicy.status to retired. New TaskAssignments and ReviewLeases then fail until another version is activated. Existing frozen work remains valid.

8.4 Explicit unpaid policy

An unpaid project still requires a published version.

```
accepted_submission:
compensation_mode: unpaid
awards: []

completed_review:
compensation_mode: unpaid
awards: []
```

This prevents missing configuration from silently removing contributor compensation.

8.5 Policy-change boundary

If version 2 becomes active:

- existing TaskAssignments retain their frozen version;
- existing ReviewLeases retain their frozen version;
- new TaskAssignments freeze version 2;
- new ReviewLeases freeze version 2;
- queued but unleased ReviewQueueEntries use the version active when a reviewer successfully claims them;
- Contribution Records always copy the version from their assignment or lease, never from the project's current setting at decision time.

---

9. Contribution Creation Matrix

Review outcome or lease result	Reviewer Contribution Record	Submitter Contribution Record	Reviewer compensation evaluated	Submitter compensation evaluated
accept	Yes	Yes	Yes	Yes
needs_revision	Yes	No	Yes	No
reject	Yes	No	Yes	No
manual release	No	No	No	No
lease expiry	No	No	No	No
administrative revocation	No	No	No	No
invalid decision attempt	No	No	No	No
idempotent replay of committed decision	Return existing	Return existing if accept	Return existing	Return existing if accept

Review compensation is decision-neutral. No award definition may match or vary by decision.

---

10. State Machines

10.1 ContributionRecord lifecycle

ContributionRecord has no mutable state machine.

```
not present --valid Review transaction--> recorded permanently
```

It cannot move to pending, failed, fulfilled, cancelled, voided, or deleted.

10.2 CompensationPolicyVersion

```
draft --publish and activate--> published --replace for new work--> retired
```

- draft may be edited;
- published is immutable and selectable for new work;
- retired is immutable and valid only through frozen references.

10.3 CompensationAward fulfillment projection

```
adapter callback: failed
+-------------------------+
| v
pending_delivery -> acknowledged pending -> failed
| | | |
| | | +--later fulfilled callback--+
| | | |
+--valid fulfilled callback-----------+--------------------------------v
fulfilled
```

Normative rules:

- Delivery acknowledgement changes only delivery_status.
- A failed callback changes fulfillment_status to failed.
- A later valid fulfilled callback changes failed to fulfilled.
- Fulfilled is terminal.
- Delivery state and fulfillment state are independent.
- A callback may arrive before local delivery acknowledgement; it is accepted if otherwise valid and MUST NOT later regress.

11. Frozen-Terms Operations

11.1 Create TaskAssignment

Preconditions:

- project has an active published CompensationPolicyVersion;
- AuthorizationService allows the upstream `task.claim` operation through an active submitter/both ProjectRoleGrant;
- task and project are claimable under existing rules.

Transaction:

```
lock project active compensation policy reference
validate version is published
create TaskAssignment
copy version id -> submitter_compensation_policy_version_id
write CompensationTermsFrozenForAssignment
commit
```

If no active version exists, assignment creation fails with compensation_policy_missing.

11.2 Create ReviewLease

This extends the atomic WS-REV-001 claim transaction.

Preconditions:

- AuthorizationService allows `review.claim` inside the claim transaction and every WS-REV-001 eligibility/capacity guard passes;
- project has an active published CompensationPolicyVersion.

Transaction addition:

```
lock project active compensation policy reference
validate version is published
create ReviewLease
copy version id -> reviewer_compensation_policy_version_id
write CompensationTermsFrozenForReviewLease
commit with queue claim
```

If the project changes policy concurrently, row locking ensures the lease freezes either the old active version before replacement commits or the new active version after replacement commits. It cannot freeze an indeterminate mixture.

11.3 Released, expired, or revoked lease

The frozen reference remains on the permanent ReviewLease attempt for audit. It creates no contribution or award because no valid Review was recorded.

---

12. Atomic Review-to-Contribution Operations

12.1 Common decision transaction

Every valid Review decision uses one PostgreSQL transaction and one unit of work.

The caller's primary permission is `review.decision.record` on the active ReviewLease. Derived creation executes through the registered fixed Workstream authorities `contribution.materialize` and `compensation.award.materialize`; these system actions do not replace or broaden the caller decision.

After all WS-REV-001 validations pass, the transaction MUST:

```
1. create immutable Review and any ReviewFinding/FindingResolution records
2. consume ReviewLease and close ReviewQueueEntry
3. apply the decision-specific Task and TaskAssignment transition
4. create reviewer ContributionRecord
5. create reviewer ContributionEvidenceProjection(status=pending)
6. write reviewer ContributionEvidenceProjectionRequested outbox event
7. evaluate the ReviewLease's frozen policy version
8. create reviewer CompensationAwards from exact completed_review definitions
9. create reviewer ContributionRecorded outbox event
10. create CompensationAwardCreated audit/outbox event for each award
11. create CompensationFulfillmentRequested outbox event for each award
12. if decision = accept:
a. create submitter ContributionRecord
b. create submitter ContributionEvidenceProjection(status=pending)
c. write submitter ContributionEvidenceProjectionRequested outbox event
d. evaluate the TaskAssignment's frozen policy version
e. create submitter CompensationAwards from exact accepted_submission definitions
f. create submitter ContributionRecorded outbox event
g. create award and fulfillment-requested events for each award
13. link the originating Review AuthorizationDecision and write all review, contribution,
compensation, task, and assignment audit events
14. commit
```

If any database insert, invariant, event write, or decision-specific transition fails, the entire transaction rolls back.

No compensation-adapter or Artifact Storage call occurs inside this transaction. External availability therefore cannot delay or invalidate the human decision or contribution recognition.

12.2 Reviewer contribution creation

Input:

- canonical Review being created;
- consumed ReviewLease;
- reviewed SubmissionVersion;
- reviewed TaskAssignment;
- frozen ReviewLease reviewer_compensation_policy_version_id.

Output:

```
contribution_type: completed_review
contributor_id: Review.reviewer_id
source_review_id: Review.id
source_review_lease_id: ReviewLease.id
source_task_assignment_id: TaskAssignment.id
submission_version_id: Review.submission_version_id
compensation_policy_version_id: ReviewLease.reviewer_compensation_policy_version_id
source_submission_artifact_digest: SubmissionVersion.artifact_digest
```

The algorithm is identical for accept, needs_revision, and reject. Branching compensation logic by Review.decision is prohibited.

12.3 Submitter contribution creation on accept

Input:

- Review with decision = accept;
- accepted SubmissionVersion;
- completed TaskAssignment;
- frozen TaskAssignment submitter_compensation_policy_version_id.

Output:

```
contribution_type: accepted_submission
contributor_id: SubmissionVersion.created_by
source_review_id: Review.id
source_review_lease_id: null
source_task_assignment_id: TaskAssignment.id
submission_version_id: Review.submission_version_id
compensation_policy_version_id: TaskAssignment.submitter_compensation_policy_version_id
source_submission_artifact_digest: SubmissionVersion.artifact_digest
```

The contribution is created only after all accept validations pass but before transaction commit.

12.4 Needs revision

Needs revision creates:

- one completed_review Contribution Record;
- zero or more reviewer CompensationAwards;
- no accepted_submission Contribution Record;
- no submitter CompensationAward.

When a later submission version is reviewed, that later Review creates another completed_review record. There is no maximum compensated review-round count in v0.1.

Structured findings, immutable chains, no-self-review, reviewer leasing, and audit events remain the controls against abusive revision cycling.

12.5 Reject

Reject creates:

- one completed_review Contribution Record;
- zero or more reviewer CompensationAwards;
- no submitter Contribution Record;
- no submitter CompensationAward.

Reviewer compensation remains valid even though the submitter TaskAssignment becomes blocked and the task closes.

12.6 Explicitly unpaid result

If the frozen policy version has no award definition for the relevant contribution type:

- create the Contribution Record;
- create ContributionRecorded;

- create no CompensationAward;
- create no CompensationFulfillmentRequested event;
- do not treat the absence of an award as an error.

The Contribution Record's policy version proves that the explicit unpaid rule was evaluated.

12.7 Integrity failure

If a frozen policy reference is missing, corrupt, from the wrong project, or contains prohibited duplicate definitions:

- reject the decision transaction with compensation_policy_integrity_error;
- create no Review or contribution;
- after rollback, write a structured integrity-error log, increment workstream_compensation_policy_integrity_errors_total, and trigger the configured operator alert;
- require operator remediation.

The implementation MUST NOT substitute the current project policy or silently treat the work as unpaid.

---

13. Compensation Evaluation

13.1 Deterministic algorithm

For one Contribution Record:

```
load exact frozen CompensationPolicyVersion
validate project match
select the one CompensationRule matching record.contribution_type
if rule is unpaid, return zero awards
if rule is compensated, load its award definitions
order definitions deterministically by instrument_type
for each definition:
copy definition fields into immutable CompensationAward
resolve the exact frozen adapter binding
create award and outbox events
return created awards
```

No scoring engine, rules interpreter, dynamic expression, network call, current reputation lookup, or current project-policy lookup is permitted.

13.2 Decision neutrality

For completed_review:

```
award(completed_review, accept)
= award(completed_review, needs_revision)
= award(completed_review, reject)
```

The Review outcome may be included as provenance in ContributionRecorded, but it cannot select or modify compensation.

13.3 Multiple instruments

If a completed-review rule defines money and project points:

```
one ContributionRecord
-> one money CompensationAward
-> one project_points CompensationAward
-> two independent fulfillment events
```

One instrument's delivery or fulfillment status never blocks or changes the other.

13.4 Decimal and unit rules

- Use an exact decimal database type.
- API quantities are decimal strings, never JSON floating-point numbers.
- Money quantities are interpreted according to unit_code but are not rounded by Workstream after policy publication.
- Policy publication rejects quantities exceeding the configured currency precision.
- Project-points quantities MUST be whole positive numbers in v0.1.
- Unit conversion is prohibited.
- Adapters cannot substitute another currency or point unit.

---

14. Transactional Event Contract

14.1 Outbox requirement

Every outbound domain or integration event is written to the existing transactional outbox in the same transaction as its source record.

The dispatcher uses at-least-once delivery. Consumers and adapters MUST implement idempotency.

No code path may:

- commit a Contribution Record and publish its event outside the outbox;
- publish an event before the database transaction commits;
- call an external adapter from the Review transaction;
- delete an outbox event because delivery is temporarily failing.

14.2 Common event envelope

```
event_id: uuid
event_type: string
event_version: 1
occurred_at: timestamp
producer: workstream

project_id: uuid
correlation_id: string
causation_event_id: uuid | null
idempotency_key: string

payload: object
```

The canonical idempotency key is stable for the logical event and does not change across delivery retries.

14.3 ContributionRecorded

One event is emitted per Contribution Record.

```
event_type: ContributionRecorded
event_version: 1
idempotency_key: "contribution:{contribution_record_id}:recorded:v1"

payload:
contribution_record_id: uuid
project_id: uuid
task_id: uuid
contributor_id: uuid
contribution_type: accepted_submission | completed_review
source_review_id: uuid
source_review_lease_id: uuid | null
source_task_assignment_id: uuid
submission_version_id: uuid
source_submission_artifact_digest: string
review_decision: accept | needs_revision | reject
compensation_policy_version_id: uuid
compensation_award_ids: list[uuid]
recorded_at: timestamp
```

Consumers may use this event for reputation signals, analytics, notifications, evidence export, or other authorized projections.

The event does not assert that compensation has been fulfilled.

14.4 ContributionEvidenceProjectionRequested

One internal projection event is emitted per Contribution Record in the same transaction that creates it.

```
event_type: ContributionEvidenceProjectionRequested
event_version: 1
idempotency_key: "contribution:{contribution_record_id}:evidence-bundle:1"

payload:
contribution_record_id: uuid
project_id: uuid
projection_schema_version: 1
requested_at: timestamp
```

The payload is a wake-up reference, not independent evidence. The worker MUST reload the canonical ContributionRecord, Review, SubmissionVersion, task policy context, ArtifactBindings, findings, resolutions, compensation policy and awards before generating the bundle.

After verified storage, Workstream emits `ContributionEvidenceProjected` with the Workstream ArtifactBinding ID and verified digest. Raw provider references MUST NOT appear in public integration events.

```
event_type: ContributionEvidenceProjected
event_version: 1
idempotency_key: "contribution:{contribution_record_id}:evidence-
projected:{projection_schema_version}"

payload:
contribution_record_id: uuid
project_id: uuid
projection_schema_version: int
artifact_binding_id: uuid
content_digest: string
projected_at: timestamp
```

14.5 CompensationAwardCreated

```
event_type: CompensationAwardCreated
event_version: 1
idempotency_key: "compensation-award:{compensation_award_id}:created:v1"

payload:
compensation_award_id: uuid
contribution_record_id: uuid
project_id: uuid
contributor_id: uuid
instrument_type: money | project_points
unit_code: string
quantity: decimal_string
compensation_policy_version_id: uuid
adapter_binding_id: uuid
created_at: timestamp
```

This event is an immutable domain fact. It is not itself the adapter instruction.

14.6 CompensationFulfillmentRequested

One event is emitted per CompensationAward.

```
event_type: CompensationFulfillmentRequested
event_version: 1
idempotency_key: "compensation-award:{compensation_award_id}:fulfill:v1"

payload:
compensation_award_id: uuid
contribution_record_id: uuid
project_id: uuid
contributor_id: uuid

instrument_type: money | project_points
unit_code: string
quantity: decimal_string

compensation_policy_version_id: uuid
adapter_binding_id: uuid
adapter_route_key: string

contribution_type: accepted_submission | completed_review
source_review_id: uuid
created_at: timestamp
```

The event contains the complete authorized instruction. The adapter MUST NOT query mutable project policy to determine a different quantity.

14.7 CompensationFulfillmentRecorded

One event is emitted for every accepted fulfillment receipt.

```
event_type: CompensationFulfillmentRecorded
event_version: 1
idempotency_key: "fulfillment-receipt:{fulfillment_receipt_id}:recorded:v1"

payload:
fulfillment_receipt_id: uuid
compensation_award_id: uuid
contribution_record_id: uuid
project_id: uuid
contributor_id: uuid
instrument_type: money | project_points
unit_code: string
authorized_quantity: decimal_string
reported_status: fulfilled | failed
external_reference: string | null
failure_code: string | null
fulfilled_at: timestamp | null
received_at: timestamp
```

This event reports Workstream's accepted receipt. It does not expose provider credentials or create a new award.

14.8 Sensitive-data restriction

Events MUST NOT contain:

- bank details;
- wallet private keys;
- provider access tokens;
- Identity Issuer bearer tokens;

- point-account credentials;
- raw personal financial data.
- raw provider CIDs, filesystem paths, or storage credentials.

The adapter resolves its own external beneficiary mapping from contributor_id through its authorized integration.

15. External Adapter Contract

15.1 Workstream adapter port

Workstream exposes one outbound port to its outbox dispatcher. Provider-specific implementations remain outside the Workstream domain.

```
class CompensationAdapterPort(Protocol):
async def deliver(
self,
binding_id: UUID,
event: CompensationFulfillmentRequested,
) -> AdapterDeliveryAcknowledgement:
...


class AdapterDeliveryAcknowledgement:
event_id: UUID
accepted: Literal[True]
acknowledged_at: datetime
```

The port implementation MUST:

- route only through the persisted adapter_binding_id;
- send the persisted event without recalculating it;
- authenticate Workstream to the adapter;
- return success only after the adapter durably stores the event idempotency key;
- translate timeouts and non-success responses into a delivery failure so the outbox retries.

The repository includes only the port, dispatcher integration, and a deterministic test adapter for the live drill. A provider-specific money or points implementation is not part of WS-CON-001.

15.2 Delivery

The outbox dispatcher routes CompensationFulfillmentRequested using the frozen adapter_binding_id and route_key.

Adapter response to delivery:

- 2xx means the adapter durably accepted the instruction and its idempotency key;
- non-2xx or timeout means delivery was not acknowledged and Workstream retries;
- a 2xx acknowledgement does not mean money or points were fulfilled.

After acknowledgement, Workstream does not create provider retries. The adapter owns all provider-side processing.

15.3 Adapter idempotency

The adapter MUST treat compensation_award_id and the event idempotency key as stable identifiers.

Repeated delivery of the same event:

- must not create another payment request;
- must not credit points twice;
- must return a successful acknowledgement after the original instruction is durably recognized.

15.4 Fulfillment callback request

```
CompensationFulfillmentReported:
external_event_id: string
compensation_award_id: uuid
status: fulfilled | failed

external_reference: string | null
fulfilled_quantity: decimal_string | null
fulfilled_at: timestamp | null

failure_code: string | null
failure_message: string | null
reported_at: timestamp
```

The callback does not accept project_id, contributor_id, instrument_type, unit_code, or authorized quantity as mutable authority. Workstream loads those from CompensationAward.

15.5 Callback authentication and authorization

Workstream MUST:

1. verify the Identity Issuer service token;
2. resolve one active service ActorProfile and active ActorIdentityLink through WS-AUTH-001;
3. load and lock the CompensationAward, ContributionRecord and frozen adapter binding from canonical identifiers;
4. ask AuthorizationService for `compensation.fulfillment.report` on the canonical award resource;
5. require the service actor to equal the frozen binding adapter_actor_id;
6. require the binding project and instrument to match the award;
7. validate callback schema and timestamps;
8. enforce idempotency before insertion;
9. persist the AuthorizationDecision reference with the callback audit chain.

The request body cannot supply project, contributor, instrument, quantity, binding or authorization scope. Those values are loaded from the award chain.

An `active` binding accepts delivery and callback work. A `suspended` binding accepts valid callbacks for already-issued awards but receives no new delivery; this permits an adapter to report a result for work it

accepted before suspension. Retirement is prohibited while an award remains unfulfilled. After retirement, only an exact replay of a previously accepted receipt may return its stored result.

The service permission is binding-derived and award-scoped. A human Admin token, contributor token, different adapter actor, disabled service actor, revoked identity link, or valid service actor without the exact frozen binding is denied.

15.6 Fulfilled callback

Validation:

- external_reference is non-empty;
- fulfilled_quantity exactly equals award.quantity;
- fulfilled_at is present and not unreasonably in the future;
- no different fulfilled receipt already exists;
- award project and binding remain internally consistent.

Effect in one transaction:

```
revalidate compensation.fulfillment.report against locked canonical award/binding
insert immutable fulfilled receipt
update status projection -> fulfilled
write CompensationFulfilled audit event linked to AuthorizationDecision
write CompensationFulfillmentRecorded outbox event
commit
```

Fulfilled is terminal.

15.7 Failed callback

Validation:

- failure_code is non-empty;
- fulfilled_quantity and fulfilled_at are null;
- no fulfilled receipt already exists.

Effect:

```
revalidate compensation.fulfillment.report against locked canonical award/binding
insert immutable failed receipt
update status projection -> failed
write CompensationFulfillmentFailed audit event linked to AuthorizationDecision
write CompensationFulfillmentRecorded outbox event
commit
```

The adapter may later report fulfilled using a new external_event_id.

15.8 Prohibited partial fulfillment

If fulfilled_quantity differs from award.quantity:

- reject with 422 partial_fulfillment_not_supported;

- create no receipt;
- do not change the projection;
- write a rejected-callback security/audit event.

The adapter must resolve partial provider behaviour internally before reporting the Workstream award as fulfilled.

15.9 Contradictory callbacks

- failed followed by fulfilled: allowed.
- failed followed by another failed with a new external_event_id: allowed.
- fulfilled followed by failed: rejected with 409 award_already_fulfilled.
- fulfilled followed by a different fulfilled event: rejected with 409 award_already_fulfilled.
- exact replay of an accepted callback: return the existing receipt with 200.
- reuse of external_event_id with different payload: reject with 409 idempotency_mismatch.

---

16. Reputation Boundary

ContributionRecorded is the canonical reputation input from this lifecycle.

Recommended raw signal mapping:

contribution_type	reputation signal
accepted_submission	submission_accepted
completed_review	review_completed

Normative rules:

- Signal creation is independent of compensation awards.
- Signal creation is independent of adapter delivery and fulfillment.
- A completed-review signal may carry Review.decision as provenance, but this specification assigns no positive or negative score based on that outcome.
- Needs revision does not automatically reduce submitter reputation.
- Reject does not create a submitter Contribution Record and therefore produces no positive submitter contribution signal.
- Lease expiry and release signals remain the separate audit events defined by WS-REV-001.
- Reputation scoring, aggregation, decay, project-to-global weighting, grant automation, and reviewer-quality adjudication are outside this specification.

The reputation consumer MUST deduplicate by contribution_record_id and event version.

---

17. Read Model and API Composition

17.1 Canonical versus composed data

The canonical Contribution Record remains unchanged after creation. Read APIs compose:

```
ContributionRecord
+ CompensationAwards
+ CompensationStatusProjection
+ latest immutable FulfillmentReceipts
+ ContributionEvidenceProjection
= Contribution detail response
```

The response may display pending, failed, or fulfilled compensation and pending, projected, or failed evidence status without storing those fields on ContributionRecord.

The read composer MUST authorize the canonical ContributionRecord before loading or disclosing awards, receipts, evidence references, or provider-derived metadata. It MUST NOT use the caller-supplied project_id as authority.

17.2 Example contribution response

```
{
"id": "contribution-uuid",
"project_id": "project-uuid",
"task_id": "task-uuid",
"contributor_id": "contributor-uuid",
"contribution_type": "completed_review",
"source_review_id": "review-uuid",
"submission_version_id": "submission-version-uuid",
"source_submission_artifact_digest": "sha256:...",
"recorded_at": "2026-07-10T12:00:00Z",
"evidence": {
"status": "projected",
"artifact_binding_id": "artifact-binding-uuid",
"content_digest": "sha256:..."
},
"compensation": [
{
"award_id": "money-award-uuid",
"instrument_type": "money",
"unit_code": "USD",
"quantity": "1.25",
"delivery_status": "acknowledged_by_adapter",
"fulfillment_status": "fulfilled",
"external_reference": "adapter-reference"
},
{
"award_id": "points-award-uuid",
"instrument_type": "project_points",
"unit_code": "REVIEW_POINT",
"quantity": "5",
"delivery_status": "acknowledged_by_adapter",
"fulfillment_status": "pending",
"external_reference": null
}
]
}
```

This is a read representation, not a mutable aggregate stored as one row.

The `artifact_binding_id` is a Workstream reference. The response does not disclose a raw provider CID, URL or filesystem path. `content_digest` proves integrity but grants no retrieval authority.

18. API Contract

The following paths are normative for Workstream v0.1. The `/v1` prefix is the independent API contract namespace, not a Workstream v1.0 release identifier.

18.1 Compensation policies

```
POST /v1/projects/{project_id}/compensation-policies
POST /v1/projects/{project_id}/compensation-policies/{policy_id}/versions
PUT /v1/projects/{project_id}/compensation-policy-versions/{version_id}/draft
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/publish
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/retire
GET /v1/projects/{project_id}/compensation-policies
GET /v1/projects/{project_id}/compensation-policy-versions/{version_id}
```

Creating a new policy creates the policy identity and an empty version 1 draft. Creating a version under an existing policy copies the current published version into the next draft version. The draft remains inactive until explicitly published.

Policy creation request:

```
{
"name": "Project default compensation"
}
```

PUT replaces the complete draft rules; it does not merge individual fields.

```
{
"rules": [
{
"contribution_type": "accepted_submission",
"compensation_mode": "compensated",
"awards": [
{
"instrument_type": "money",
"unit_code": "USD",
"quantity": "45.00",
"adapter_binding_id": "money-binding-uuid"
}
]
},
{
"contribution_type": "completed_review",
"compensation_mode": "compensated",
"awards": [
{
"instrument_type": "money",
"unit_code": "USD",
"quantity": "60.00",
"adapter_binding_id": "money-binding-uuid"
},
{
"instrument_type": "project_points",
"unit_code": "REVIEW_POINT",
"quantity": "5",
"adapter_binding_id": "points-binding-uuid"
}
]
}
]
}
```

The publish request and the retire request both require:

```
{
"expected_current_version_id": "current-version-uuid-or-null"
}
```

No edit endpoint exists for a published or retired version.

Policy routes use exactly one primary permission: `compensation.policy.create_draft`, `compensation.policy.update_draft`, `compensation.policy.publish`, `compensation.policy.retire`, or `compensation.policy.read`, matched to the operation and canonical target. Every write requires Finance Authority. Possession of a route path is not authority.

18.2 Adapter bindings

```
POST /v1/projects/{project_id}/compensation-adapter-bindings
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/suspend
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/resume
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/retire
GET /v1/projects/{project_id}/compensation-adapter-bindings
```

Binding creation request:

```
{
"instrument_type": "money",
"adapter_actor_id": "service-actor-uuid",
"route_key": "project-money-v1"
}
```

The route_key identifies deployment routing configuration and contains no endpoint credential or secret.

Binding routes use exactly one primary permission: `compensation.adapter_binding.create`, `compensation.adapter_binding.suspend`, `compensation.adapter_binding.resume`, `compensation.adapter_binding.retire`, or `compensation.adapter_binding.read`, matched to the operation and canonical target. Every mutation requires Finance Authority. Operational views use `operations.status.read`, `compensation.award.read`, or `audit.read` according to the canonical view.

18.3 Contributions

```
GET /v1/contributors/me/contributions
GET /v1/contributors/me/contributions/{contribution_record_id}
GET /v1/contributors/me/contributions/{contribution_record_id}/evidence
GET /v1/projects/{project_id}/contributions
GET /v1/projects/{project_id}/contributions/{contribution_record_id}
GET /v1/projects/{project_id}/contributions/{contribution_record_id}/evidence
GET /v1/projects/{project_id}/contributors/{contributor_id}/contributions
GET /v1/projects/{project_id}/tasks/{task_id}/contributions
```

Supported filters:

- project_id;
- contribution_type;
- task_id;
- recorded_from;

- recorded_to;
- cursor.

Ordering is deterministic:

```
recorded_at DESC, id DESC
```

Authorization is deterministic:

- `GET /v1/contributors/me/contributions` requires `contribution.read_self` and returns only records whose contributor_id is the caller's ActorProfile id;
- the two `/contributors/me/...` detail/evidence routes use `contribution.read_self` and require the record owner to equal the caller;
- every `/projects/{project_id}/...` contribution route uses `contribution.read_project` and derives exact scope from the canonical Project/ContributionRecord chain;
- project-scoped contributor history uses `/projects/{project_id}/contributors/{contributor_id}/contributions`; no cross-project Admin contributor-history route exists in v0.1;
- project and task endpoints require `contribution.read_project` for the canonical path project scope;
- records from unauthorized projects are never returned and are not revealed through total counts.

No POST, PATCH, PUT, or DELETE Contribution Record endpoint exists.

The self and project evidence endpoints apply their one declared contribution-read permission, validate the projection and ArtifactBinding chain, retrieve through ArtifactStorePort with a Workstream service scope, verify the digest, record attributable access and return the approved bundle representation.

Responses:

- `200` when the verified bundle is available;
- `409 contribution_evidence_not_ready` while projection is pending;
- `503 contribution_evidence_unavailable` when projection or verified retrieval has failed;
- `404 contribution_not_found` when the record is absent or invisible.

18.4 Compensation awards

```
GET /v1/compensation-awards/{award_id}
GET /v1/contributors/me/compensation-awards
GET /v1/projects/{project_id}/compensation-awards
GET /v1/projects/{project_id}/contributors/{contributor_id}/compensation-awards
POST /v1/projects/{project_id}/compensation-awards/{award_id}/reconcile-delivery
```

Supported operational filters:

- instrument_type;
- delivery_status;
- fulfillment_status;
- created_from;
- created_to;

- minimum_pending_age.

Award reads require `compensation.award.read` for the canonical award or constrained collection. A beneficiary contributor may read only their own awards under the subject guard; project readers see only covered project rows. No cross-project Admin contributor-award route exists in v0.1. The contribution endpoint may also compose the owner's authorized award summary without granting broader award visibility.

The reconciliation route requires `compensation.delivery.reconcile`, a mandatory `Idempotency-Key` header and a bounded structured reason. It may inspect delivery evidence or make the existing unacknowledged outbox instruction immediately eligible for dispatch with the same event_id, payload and idempotency key. It cannot create an award, change quantity/binding, fabricate acknowledgement, or mark fulfillment. A second equivalent request returns the stored result; an incompatible live request returns `409 compensation_reconciliation_in_progress`.

18.5 Fulfillment callback

```
POST /v1/integrations/compensation/fulfillment-reports
```

Responses:

- 201 when a new receipt is created;
- 200 for exact idempotent replay;
- 400 for malformed schema;
- 401 for invalid identity;
- 403 for unauthorized adapter or project/instrument mismatch;
- 404 for unknown award;
- 409 for callback conflict or idempotency mismatch;
- 422 for invalid fulfillment semantics.

No endpoint allows an adapter to change the Contribution Record, CompensationAward, policy, assignment, lease, Review, task, or contributor.

The callback requires an Identity Issuer service token and an allowed `compensation.fulfillment.report` AuthorizationDecision for the exact frozen award binding. Human Admin permissions cannot satisfy this route.

19. Error Contract

Errors MUST use the existing Workstream structured-error envelope and stable machine codes.

Code	HTTP	Meaning
compensation_policy_missing	409	Project has no active published compensation version
compensation_policy_not_draft	409	Attempt to edit non-draft version
compensation_policy_invalid	422	Draft violates policy rules
compensation_policy_integrity_error	500	Frozen reference or published data is internally inconsistent
authentication_required	401	Required bearer token is absent
invalid_token	401	Identity Issuer token verification failed

Code	HTTP	Meaning
actor_not_active	403	ActorProfile or identity link is not active
permission_not_granted	403	AuthorizationService denied the registered permission
resource_project_mismatch	403	Canonical record chain does not belong to the covered project scope
service_actor_not_provisioned	403	Callback identity is not an active provisioned service actor
adapter_binding_missing	409	Required instrument has no compatible binding
adapter_binding_in_use	409	Binding is referenced by active policy, unfinished frozen work, or unfulfilled awards
adapter_unauthorized	403	Callback actor is not bound to the award
compensation_reconciliation_in_progress	409	An incompatible live delivery-reconciliation request already exists for the award/event
contribution_not_found	404	Contribution Record does not exist or is not visible
compensation_award_not_found	404	Award does not exist or is not visible
contribution_already_recorded	409	Logical contribution already exists
compensation_award_already_exists	409	Logical award already exists
fulfillment_callback_invalid	422	Callback fields do not match status rules
partial_fulfillment_not_supported	422	Fulfilled quantity differs from authorized quantity
award_already_fulfilled	409	A contradictory post-fulfillment callback was attempted
idempotency_mismatch	409	Idempotency identifier was reused with different payload
contribution_evidence_not_ready	409	Contribution evidence projection has not completed
contribution_evidence_unavailable	503	Verified contribution evidence cannot currently be retrieved
artifact_binding_not_visible	404	ArtifactBinding does not exist in the caller's authorized contribution scope
artifact_integrity_mismatch	500	Stored evidence receipt or bytes do not match the expected digest or source identity
artifact_idempotency_conflict	409	Stable artifact operation identity was reused with different bytes or scope
recovery_in_progress	409	One live recovery attempt already exists for the artifact outbox operation

Internal uniqueness races MUST be translated into the corresponding stable conflict or idempotent success rather than leaking database errors.

Authentication, authorization, domain-state, adapter-binding, and Artifact Storage failures MUST retain distinct codes. The API MUST NOT translate every denial into `not_found` or every dependency failure into a generic 500.

20. Database Constraints and Indexes

Required constraints:

```
ContributionRecord:
UNIQUE(source_review_id, contribution_type)
CHECK(contribution_type IN accepted_submission, completed_review)
source_submission_artifact_digest NOT NULL

ContributionEvidenceProjection:
PRIMARY KEY(contribution_record_id)
CHECK(status IN pending, projected, failed)
CHECK(projected requires artifact_binding_id, content_digest, projected_at)
CHECK(pending or failed forbids projected_at)

CompensationPolicyVersion:
UNIQUE(compensation_policy_id, version_number)

CompensationPolicy:
partial UNIQUE(project_id) WHERE status = active

CompensationRule:
UNIQUE(compensation_policy_version_id, contribution_type)
CHECK(compensation_mode IN compensated, unpaid)

CompensationAwardDefinition:
UNIQUE(compensation_policy_version_id, contribution_type, instrument_type)
CHECK(quantity > 0)

CompensationAward:
UNIQUE(contribution_record_id, instrument_type)
CHECK(quantity > 0)

ProjectCompensationAdapterBinding:
partial UNIQUE(project_id, instrument_type) WHERE status = active

CompensationFulfillmentReceipt:
UNIQUE(adapter_binding_id, external_event_id)
partial UNIQUE(compensation_award_id) WHERE reported_status = fulfilled

CompensationStatusProjection:
PRIMARY KEY(compensation_award_id)

TaskAssignment:
submitter_compensation_policy_version_id NOT NULL

ReviewLease:
reviewer_compensation_policy_version_id NOT NULL

Shared ArtifactOutboxRecoveryAttempt:
partial UNIQUE(outbox_id) WHERE status IN (not_started, reconciling)
```

The generic ArtifactBinding tables MUST enforce one verified `contribution_evidence_bundle` binding per `(contribution_record_id, projection_schema_version)` and the same project_id as the owner ContributionRecord. An ArtifactBinding from another owner, project or artifact role cannot satisfy a projection.

Foreign keys MUST prevent project-chain mismatches. Composite foreign keys MUST be used where a single-column foreign key cannot express project ownership. Transaction-level validation remains additional defence and MUST NOT replace enforceable database ownership constraints. AuthorizationDecision references on sensitive audit/mutation records MUST point to an immutable WS-AUTH decision; bearer tokens are never stored.

Required indexes:

```
ContributionRecord(contributor_id, recorded_at DESC, id DESC)
ContributionRecord(project_id, recorded_at DESC, id DESC)
ContributionRecord(project_id, task_id, recorded_at, id)
ContributionRecord(source_review_id, contribution_type)
ContributionEvidenceProjection(project_id, status, updated_at)
ContributionEvidenceProjection(status, last_attempt_at)

CompensationPolicyVersion(project_id, status)
CompensationAward(contribution_record_id)
CompensationAward(project_id, created_at DESC)
CompensationAward(adapter_binding_id, created_at)

CompensationFulfillmentReceipt(compensation_award_id, received_at)
CompensationStatusProjection(fulfillment_status, updated_at)

Outbox(event_type, delivery_state, next_attempt_at)
```

All timestamps are stored in UTC.

21. Concurrency and Idempotency

21.1 Review-decision replay

The WS-REV-001 decision idempotency key protects the entire combined transaction.

Repeating the same committed decision request MUST return:

- the existing Review;
- the existing reviewer Contribution Record;
- the existing submitter Contribution Record when decision = accept;
- the existing CompensationAwards;
- no new outbox events.

Reusing the decision idempotency key with a different decision or payload returns 409 idempotency_mismatch.

21.2 Concurrent decision attempts

If two requests attempt to decide the same ReviewLease or SubmissionVersion:

- exactly one may create the Review;
- exactly one completed_review Contribution Record exists;
- at most one accepted_submission Contribution Record exists;
- award uniqueness follows the winning contribution records;
- the losing transaction returns the existing result only if it is an exact idempotent replay; otherwise it returns a conflict.

21.3 Concurrent policy activation

Project policy activation locks the project active-version reference.

If two versions are activated concurrently:

- exactly one becomes active first;
- the other transaction must re-evaluate and either replace it explicitly under its request semantics or fail with a version conflict;
- assignments and leases freeze one committed version, never a mixture.

Activation endpoints MUST require expected_current_version_id. Null is supplied only when the project has no active version.

21.4 Policy activation versus assignment

Assignment and lease creation lock the same project active-version reference used by activation.

The result is serializable at that boundary:

- work freezes the old version before activation commits; or
- work freezes the new version after activation commits.

Server request timing outside the transaction does not determine the version.

21.5 Award duplication

CompensationAward creation is part of the contribution transaction and protected by uniqueness constraints.

An insert conflict caused by an exact replay resolves to the existing award. A conflict with different unit, quantity, policy version, or binding is an integrity error.

21.6 Outbox delivery replay

Outbox delivery is at-least-once.

- The event_id and idempotency_key remain unchanged across retries.
- Delivery-attempt count and next-attempt time are operational metadata.
- A transient adapter failure does not create another CompensationAward or outbox event.
- After durable adapter acknowledgement, provider-side retries belong to the adapter.

21.7 Concurrent callbacks

Callback processing locks the CompensationAward or its status row.

If failed and fulfilled callbacks race:

- if fulfilled commits first, the failed callback is rejected;
- if failed commits first, the fulfilled callback may subsequently commit;
- final state is fulfilled.

If two different fulfilled callbacks race, exactly one may create the unique fulfilled receipt. The other is rejected unless it is the exact idempotent replay of the winner.

21.8 Callback before delivery acknowledgement

A valid authenticated callback may be received before Workstream records local delivery acknowledgement.

In that case:

- accept and store the receipt;
- set fulfillment_status according to the receipt;
- set delivery_status to acknowledged_by_adapter because possession of the award instruction is proven;
- mark the associated outbox instruction delivered or suppress further delivery idempotently;
- never regress fulfilled state when a late dispatcher acknowledgement arrives.

21.9 Authorization and mutation races

Policy publication, binding mutation, outbox retry, projection rebuild, reconciliation and callback processing MUST evaluate AuthorizationService against canonical resources after those resources are loaded. Sensitive write transactions revalidate the decision after acquiring the rows whose state determines authority or domain validity.

If a grant, service ActorProfile, identity link or binding becomes invalid before the write commits, the operation fails closed. A previously allowed router decision is not a perpetual capability and cannot authorize a later transaction after revocation.

21.10 Contribution-evidence projection replay

Decision replay returns the existing ContributionEvidenceProjection row and MUST NOT create another projection event.

Projection delivery is at-least-once:

- the worker always reloads canonical data;
- the same contribution id and projection schema version produce the same canonical serialization and digest;
- retry uses the same artifact operation identity and idempotency key;
- the same identity and digest resolves to the existing verified ArtifactBinding;
- the same identity with different bytes, owner, project, media type or scope is an integrity conflict;
- concurrent workers may produce at most one verified current projection for the contribution/schema version.

An authorized rebuild to a later projection schema version creates a new deterministic storage identity and retains the prior ArtifactBinding for audit. It never rewrites an existing bundle.

21.11 Uncertain artifact-operation recovery

Provider call attempts and provider logical effects are counted separately. A timeout may require receipt lookup or exact replay, while the stable provider idempotency identity permits at most one logical stored bundle or retention effect.

Only one live recovery attempt may exist for an artifact outbox operation. An Operator authorizes recovery with `artifact.recovery.request`; a Celery worker bound to `artifact.recovery.execute` owns the qualified artifact-recovery execution lease. A worker crash increments the execution generation on the same attempt so a stale worker cannot overwrite the newer result. Operators never take over each other's attempts.

22. Background Processing and Recovery

22.1 Outbox dispatcher

The dispatcher:

- selects committed undelivered events;
- routes by adapter_binding_id and route_key;
- sends the exact persisted payload;
- records delivery attempts;
- retries transient failures using delay = min(300 seconds, 2 raised to attempt_number minus 1 seconds), plus uniformly distributed jitter from zero through ten percent of that delay;
- marks acknowledged only after a valid adapter 2xx response;
- preserves events until retention policy permits archival.

It MUST NOT derive a new award quantity from current policy.

There is no maximum delivery-attempt count in v0.1. After ten consecutive failed attempts, the dispatcher continues retrying and triggers the delayed-delivery alert.

22.2 Contribution-evidence projection worker

The worker consumes `ContributionEvidenceProjectionRequested` after commit and executes section 6.13 through ArtifactStorePort. It runs under a provisioned Workstream system ActorProfile and uses least-privilege service scopes; it never impersonates the contributor or forwards the initiating human token.

The worker MUST:

- claim projection work idempotently;
- reload canonical records and authorized ArtifactBindings;
- serialize the versioned bundle deterministically;
- compute and retain the expected digest before storage;
- verify the provider receipt before creating the bundle ArtifactBinding;
- apply the configured evidence retention class;
- update the projection and audit/outbox records atomically after verified success;
- distinguish retryable availability failure, permanent canonical-integrity failure and uncertain provider outcome.

An indexing capability, when configured, consumes the verified bundle only after storage. Indexing failure does not invalidate the stored bundle.

22.3 Compensation status projection updater

Projection updates happen synchronously in the same transaction that records adapter delivery acknowledgement or a fulfillment receipt. Projection rebuild remains an asynchronous recovery operation.

The updater:

- updates are idempotent;
- fulfilled cannot regress;
- the projection can be fully rebuilt;
- source receipts and delivery records remain authoritative.

22.4 Reconciliation job

The reconciliation job detects:

- Review without required reviewer Contribution Record;
- accept Review without submitter Contribution Record;
- Contribution Record without ContributionRecorded outbox event;
- Contribution Record whose frozen policy version does not match its assignment or lease;
- award missing for a published definition;
- unexpected award for an unpaid contribution type;
- award without CompensationFulfillmentRequested;
- fulfilled projection without fulfilled receipt;
- fulfilled receipt whose quantity differs from award;
- multiple fulfilled receipts;
- adapter binding retired while referenced by an unfulfilled award;
- acknowledged delivery whose adapter binding does not match the award.
- Contribution Record without a pending or projected ContributionEvidenceProjection;
- Contribution Record without a ContributionEvidenceProjectionRequested outbox event;
- projected evidence row without one verified contribution_evidence_bundle ArtifactBinding;
- evidence bundle whose owner, project, source identity or digest differs from the canonical chain;
- projected bundle whose required retention operation is absent or failed;
- a raw provider reference exposed by a public contribution read model.

Reconciliation MUST alert operators and use explicit replay or compensating operational actions. It MUST NOT silently edit immutable Reviews, Contribution Records, awards, definitions, or receipts.

22.5 Event replay

Safe replay is allowed for:

- rebuilding read projections;
- recreating a missing delivery attempt from an existing outbox event;

- re-delivering an unacknowledged event with the same identifiers;
- recreating a missing mutable status projection from authoritative records.
- re-running contribution evidence projection from the canonical ContributionRecord chain with the same schema version and operation identity;
- rebuilding a later evidence schema version under an authorized projection-rebuild operation.

Replay MUST NOT create a second logical Contribution Record, award, fulfilled receipt, or bundle for the same contribution/schema version.

22.6 Artifact recovery

An uncertain artifact operation becomes `dead_letter` after its bounded automatic retry policy is exhausted. Automatic execution then stops until an Operator requests reconciliation.

```
Operator recovery request
-> one append-only recovery attempt
-> Celery worker claims artifact-recovery execution generation
-> provider receipt lookup or exact idempotent replay
-> recovered, confirmed_no_effect, safely_requeued, or unsafe_ambiguous
```

PostgreSQL MUST enforce one live recovery attempt per outbox operation with a partial unique constraint over `not_started` and `reconciling`, in addition to locking the outbox row during creation. A second non-idempotent request while recovery is live returns `409 recovery_in_progress` and makes no mutation.

The Operator is the immutable requester, not the lease owner. Celery worker identity, lease expiry and execution generation are internal qualified fields. No interrupted state or cross-Operator takeover exists. The original provider idempotency identity is always reused.

The shared recovery attempt therefore retains `requester_ref` and uses the qualified executor fields `artifact_recovery_executor_id`, `artifact_recovery_lease_expires_at`, and `artifact_recovery_execution_generation`. Generic fields such as `owner`, `claim_owner`, or `lease_owner` are prohibited because they can be confused with contributor/reviewer ownership.

22.7 Adapter suspension

When a binding is suspended:

- existing awards remain valid;
- undelivered events remain pending;
- no award is cancelled;
- Review decisions and contribution creation continue if the policy version was already frozen;
- new policy publication cannot select the suspended binding;
- new TaskAssignments or ReviewLeases cannot freeze a current policy that depends on a suspended binding.

Existing assignments and leases are honored because contributors began work under frozen terms. Delivery resumes when the same logical binding is reactivated.

23. Audit Events

The following event types are required.

Policy

- CompensationPolicyCreated
- CompensationPolicyVersionDrafted
- CompensationPolicyVersionPublished
- CompensationPolicyVersionActivated
- CompensationPolicyVersionRetired
- CompensationAwardDefinitionCreated
- ProjectCompensationAdapterBound
- ProjectCompensationAdapterSuspended
- ProjectCompensationAdapterResumed
- ProjectCompensationAdapterRetired

Frozen terms

- CompensationTermsFrozenForAssignment
- CompensationTermsFrozenForReviewLease

Contribution

- ReviewerContributionRecorded
- SubmitterContributionRecorded
- ContributionRecorded
- ContributionEvidenceProjectionRequested
- ContributionEvidenceProjected
- ContributionEvidenceProjectionFailed
- ContributionEvidenceProjectionReconciled
- ContributionEvidenceAccessed
- ContributionEvidenceUnavailable
- ContributionEvidenceIntegrityMismatch

Award and delivery

- CompensationAwardCreated
- CompensationFulfillmentRequested
- CompensationFulfillmentDeliveryAttempted
- CompensationFulfillmentDeliveryAcknowledged
- CompensationFulfillmentDeliveryFailed

Callback

- CompensationFulfillmentReported
- CompensationFulfilled
- CompensationFulfillmentFailed
- CompensationFulfillmentCallbackRejected
- CompensationFulfillmentCallbackReplayed

Artifact recovery

- ContributionArtifactRecoveryRequested
- ContributionArtifactRecoveryStarted
- ContributionArtifactRecovered
- ContributionArtifactConfirmedNoEffect
- ContributionArtifactSafelyRequeued
- ContributionArtifactRecoveryUnsafeAmbiguity

Every audit event records:

```
event_id: uuid
event_type: string
occurred_at: timestamp
actor_id: actor_id | system_actor_id
authorization_decision_id: uuid | null

project_id: uuid
contributor_id: uuid | null
task_id: uuid | null
review_id: uuid | null
review_lease_id: uuid | null
contribution_record_id: uuid | null
compensation_award_id: uuid | null
compensation_policy_version_id: uuid | null
adapter_binding_id: uuid | null
artifact_binding_id: uuid | null
artifact_operation_id: uuid | null
artifact_recovery_attempt_id: uuid | null

correlation_id: string
causation_event_id: uuid | null
reason_code: string | null
metadata: object
```

Audit metadata MUST redact credentials and sensitive financial data.

Sensitive human and service mutations record the exact WS-AUTH AuthorizationDecision id, permission, canonical scope and policy version by reference. Audit records MUST NOT persist bearer tokens, Artifact Storage service credentials, raw provider URLs, or full evidence-bundle contents.

---

24. Notifications and Operational Views

Notification transport is an adapter concern. Workstream emits events sufficient to notify:

- contributor that a submission contribution was recorded;
- reviewer that a review contribution was recorded;
- contributor of the compensation instruments authorized;
- Admin that adapter delivery is delayed;
- contributor or Admin that fulfillment was reported failed;
- contributor that fulfillment was reported completed;
- Admin that policy or adapter binding is missing or suspended.
- authorized Operator that a contribution-evidence projection or retention operation requires recovery.

24.1 Contributor view

Must show:

- global contribution history;
- project filter;
- contribution type and source task;
- recorded time;
- compensation awarded per instrument;
- pending, failed, or fulfilled status from the projection;
- contribution-evidence projection status and an authorized Workstream evidence reference when projected;
- external reference only when permitted by data policy.

It MUST NOT imply that adapter acknowledgement means fulfillment.

24.2 Project Admin view

Must show:

- active compensation version;
- historical frozen versions;
- assignment and lease counts by frozen version;
- contribution counts by type;
- awards by instrument and unit;
- oldest pending-delivery age;
- oldest pending-fulfillment age;
- failed fulfillment count and failure codes;
- adapter binding health and suspension state.
- contribution-evidence projection completion/failure counts and oldest pending age.

24.3 Finance Authority view

Within authorized project scope, Finance Authority may read:

- monetary award totals;
- monetary awards by fulfillment status;
- policy versions containing money;
- money-adapter delivery health;
- external references returned by the authorized adapter.

Workstream does not provide provider balances, payout batches, or settlement ledgers.

24.4 Audit Authority and authorized reputation-consumer view

Within covered scope, Audit Authority may read the attributable contribution, authorization, compensation and evidence-projection chain. An external reputation service consumes only the authorized `ContributionRecorded` event contract; it is a service consumer, not a human Admin role.

The allowed view includes:

- Contribution Records;
- contribution types;
- source Reviews and provenance;
- ContributionRecorded event delivery status to the reputation consumer.
- contribution-evidence projection status and audit references, without raw provider authority.

Compensation fulfillment is not a reputation input.

Every operational and aggregate view is filtered after AuthorizationService evaluates the canonical project/system scope. Unauthorized rows are excluded before counts, totals, oldest-age calculations or failure-code aggregation.

25. Observability

Required metrics:

```
workstream_contributions_created_total{project_id, contribution_type}
workstream_contribution_transaction_failures_total{reason}
workstream_compensation_awards_created_total{project_id, instrument_type, unit_code}
workstream_compensation_award_quantity_total{project_id, instrument_type, unit_code}
workstream_compensation_outbox_pending_total{adapter_binding_id}
workstream_compensation_outbox_oldest_seconds{adapter_binding_id}
workstream_compensation_delivery_attempts_total{adapter_binding_id, result}
workstream_compensation_fulfillment_callbacks_total{adapter_binding_id, status, result}
workstream_compensation_pending_fulfillment_total{project_id, instrument_type}
workstream_compensation_pending_fulfillment_oldest_seconds{project_id, instrument_type}
workstream_compensation_reconciliation_violations_total{type}
workstream_contribution_authorization_decisions_total{permission, result, reason_code}
workstream_contribution_evidence_projection_total{result, error_code}
workstream_contribution_evidence_projection_lag_seconds{project_id}
workstream_contribution_evidence_retrieval_total{result}
workstream_contribution_evidence_unavailable_total{reason}
workstream_contribution_evidence_integrity_mismatch_total{type}
workstream_contribution_artifact_recovery_total{result}
```

Required tracing attributes:

- correlation_id;
- project_id;
- task_id;
- review_id;
- contribution_record_id;
- compensation_award_id;
- adapter_binding_id;
- event_id;
- authorization_decision_id;
- artifact_binding_id;
- artifact_operation_id;
- projection_schema_version.

Logs MUST NOT contain bearer tokens, credentials, private keys, bank details, or full provider payloads containing sensitive data.

Suggested alerts:

- outbox oldest age exceeds project operational threshold;
- adapter delivery failure rate exceeds threshold;
- fulfillment callback rejection spikes;
- reconciliation detects any missing contribution or duplicate fulfilled receipt;
- an active policy loses a usable adapter binding;
- monetary awards remain pending beyond project service target;
- contribution-evidence projection lag exceeds the configured service target;
- any contribution-evidence integrity mismatch occurs;
- any artifact recovery ends in unsafe ambiguity;
- authorization denials or callback-actor mismatches spike unexpectedly.

26. Security Requirements

1. Verify Identity Issuer tokens and resolve the current ActorProfile/identity link on every protected human and service request.
2. Use WS-AUTH-001 AuthorizationService for every registered action; local role-name conditionals are prohibited.
3. Derive project and record scope from canonical Workstream rows, not request paths, query filters, event payloads or callback fields.
4. Require Finance Authority and the exact granular `compensation.policy.*` action from WS-AUTH-001-A for every compensation-policy mutation, whether money, project points or unpaid.
5. Require Finance Authority and the exact granular `compensation.adapter_binding.*` action from WS-AUTH-001-A for every adapter-binding mutation.

6. Never allow a human actor to create, edit, fulfill or delete ContributionRecords, CompensationAwards or fulfillment receipts.
7. Bind callbacks to an active service ActorProfile, `compensation.fulfillment.report`, and the exact frozen adapter actor, project, instrument and award.
8. Treat callback and external adapter payloads as untrusted input.
9. Use exact decimal parsing with bounded length and precision.
10. Reject reported_at or fulfilled_at more than five minutes ahead of database time. A fulfilled_at value must not be more than five minutes later than reported_at.
11. Enforce request-body size limits and rate-limit callback endpoints per adapter actor.
12. Store adapter and Artifact Storage credentials outside domain tables.
13. Sign or mutually authenticate adapter delivery in addition to service identity where deployment policy requires it.
14. Prevent insecure direct-object references in contribution, award, receipt, audit and evidence endpoints.
15. Re-authorize every Artifact Storage search candidate against its canonical ContributionRecord before disclosure.
16. Human bearer tokens MUST NOT cross the Workstream boundary. Artifact Storage and compensation adapters receive separate service credentials/scopes.
17. Artifact Storage service scopes MUST be least-privilege and bind project, owner, operation, artifact role and expiry where the provider supports those fields.
18. Verify contribution-evidence digest, source identity, project ownership, artifact role, media type and operation identity before creating an ArtifactBinding.
19. Do not expose artifact bytes through list endpoints or expose raw CIDs, arbitrary URLs, filesystem paths, credentials or provider capabilities as authority.
20. Redact external failure messages before showing them to contributors.
21. Preserve immutable, attributable audit evidence for authorization, policy publication, award creation, delivery, callbacks, evidence projection, access and recovery.
22. Treat money and project-points fulfillment events as economically sensitive messages.
23. Treat contribution evidence as project-confidential unless the project's explicit disclosure policy grants a broader authorized view.
24. A storage or adapter outage MUST fail the dependent projection/delivery operation without weakening authorization or mutating canonical contribution history.

27. End-to-End Reference Sequences

27.1 Accepted submission with paid reviewer and paid submitter

```
1. TaskAssignment created under CompensationPolicyVersion P1
- submitter terms frozen to P1
2. Submission v1 passes checkers and enters review queue
3. Reviewer claims under current policy P1
- ReviewLease terms frozen to P1
4. Reviewer records accept; AuthorizationService allows `review.decision.record` for the
locked lease/task scope
5. One transaction:
- Review accept created
- lease consumed and queue closed
- Task.status -> accepted
- TaskAssignment.status -> completed
- reviewer completed_review ContributionRecord created
- reviewer awards created from P1 completed_review rules
- submitter accepted_submission ContributionRecord created
- submitter awards created from P1 accepted_submission rules
- one pending ContributionEvidenceProjection and projection event per ContributionRecord
- contribution, award, fulfillment and projection outbox events created
- commit
6. Evidence worker builds deterministic reviewer/submitter bundles and stores them through
ArtifactStorePort
7. Adapter deliveries occur after commit
8. Money and points adapters acknowledge independently
9. Each bound service actor is authorized and later reports fulfilled or failed
10. Workstream stores receipts and updates read projections
```

27.2 Needs revision with reviewer compensation

```
1. Reviewer records needs_revision with required findings
2. One transaction:
- Review created
- task -> needs_revision
- reviewer completed_review ContributionRecord created
- reviewer awards created independent of decision
- no submitter Contribution Record
- commit
3. Contributor submits v2
4. v2 is reviewed later
5. That later valid Review creates another reviewer Contribution Record
```

27.3 Reject with reviewer compensation

```
1. Reviewer records reject
2. One transaction:
- Review reject created
- reviewer completed_review ContributionRecord and awards created
- submitter TaskAssignment blocked
- task closed
- no submitter Contribution Record
- commit
3. Reviewer compensation proceeds through external adapter
```

27.4 Explicitly unpaid project

```
1. Project has active published unpaid policy
2. Reviewer records accept
3. Reviewer and submitter Contribution Records are created
4. No CompensationAwards are created
5. ContributionRecorded events still feed reputation and history
```

27.5 Money plus project points

```
completed_review ContributionRecord
-> money award USD 1.25 -> money adapter
-> project points award REVIEW_POINT 5 -> points adapter

money callback -> fulfilled
points callback -> failed

read view:
money = fulfilled
points = failed
contribution remains permanently recorded
```

27.6 Adapter failure followed by success

```
1. Adapter acknowledges award instruction
2. Adapter reports failed with external_event_id F1
3. Workstream stores failed receipt; projection -> failed
4. Adapter retries provider work internally
5. Adapter reports fulfilled with external_event_id F2
6. Workstream stores fulfilled receipt; projection -> fulfilled
7. A later failed callback is rejected
```

27.7 Policy changes during existing work

```
1. TaskAssignment freezes P1
2. Reviewer R1 claims and freezes P1
3. Project activates P2
4. R1 completes review:
- reviewer award uses P1
- accepted submitter award uses TaskAssignment P1
5. Reviewer R2 claims later:
- R2 lease freezes P2
6. No existing object is rewritten
```

27.8 Duplicate decision and duplicate callback

```
1. Decision request commits Review, contributions, awards, and events
2. Same idempotency key and payload is retried
3. Existing complete result is returned; no duplicate records
4. Adapter callback commits one receipt
5. Same external_event_id and payload is retried
6. Existing receipt is returned with 200
```

27.9 Authorized contribution-evidence retrieval

```
1. Caller requests the self or project-scoped contribution evidence route with Identity
Issuer token
2. AuthorizationService allows contribution.read_self or contribution.read_project on
canonical record
3. Workstream validates the projected ArtifactBinding owner, project, role and digest
4. Workstream retrieves through ArtifactStorePort using a separate least-privilege service
scope
5. Digest is verified and attributable access is recorded
6. Approved bundle representation is returned
7. Raw provider authority and the caller token are never forwarded or exposed
```

27.10 Storage outage and uncertain recovery

```
1. Review transaction commits Review, contributions, awards and pending evidence projections
2. Artifact Storage is unavailable; projection retry fails
3. Review, ContributionRecords and CompensationAwards remain valid
4. An uncertain provider operation reaches dead_letter and automatic execution stops
5. Authorized Operator requests artifact recovery once
6. Celery worker resolves provider receipt or exactly replays the original idempotent
operation
7. Recovery either projects the verified bundle, confirms no effect/safe requeue, or records
unsafe ambiguity
8. No Operator owns an execution lease and no canonical lifecycle fact is rewritten
```

28. Conformance Tests

An implementation is not conformant until all applicable tests pass against PostgreSQL and the real API transaction boundary.

28.1 Policy tests

1. Project without an active policy cannot create a TaskAssignment.
2. Project without an active policy cannot create a ReviewLease.
3. Explicit unpaid policy allows work and creates no awards.
4. Draft policy may be edited by Finance Authority with an allowed `compensation.policy.update_draft` decision for the canonical version.
5. Published policy cannot be edited.
6. Retired policy cannot be edited.
7. Money policy cannot be published by Project Manager, Operator, Access Administrator, Audit Authority or Contributor.
8. Points-only and explicitly unpaid policies have the same Finance Authority requirement; Project Manager cannot publish them.
9. Duplicate money definition for one contribution type is rejected.
10. Duplicate points definition for one contribution type is rejected.
11. Zero or negative quantity is rejected.
12. Floating-point JSON quantity is rejected when the API requires a decimal string.
13. Invalid currency code is rejected.
14. Fractional project points are rejected.
15. Definition using another project's binding is rejected.
16. Dynamic outcome, contributor, task, skill, tier, or reputation condition is rejected.
17. Concurrent activation produces one deterministic active version.

17a. Router-level role state without a transaction-time AuthorizationDecision cannot publish or retire a version.

17b. Revoked Finance Authority fails closed before policy mutation commits.

17c. Cross-project policy or adapter-binding path substitution is denied from canonical ownership.

17d. Finance Authority can create, suspend, resume and retire a valid project binding; Project Manager and Operator cannot.

28.2 Freeze tests

18. TaskAssignment stores the active version at creation.
19. ReviewLease stores the active version at claim.
20. Later activation does not change existing assignment.
21. Later activation does not change existing lease.
22. New assignment after activation uses the new version.
23. New lease after activation uses the new version.
24. Retired frozen version remains usable at decision time.
25. Suspended binding prevents new work from freezing a dependent current policy.
26. Existing frozen work remains recognizable while binding is suspended.

26a. TaskAssignment freeze is reached only after the upstream `task.claim` authorization succeeds.

26b. ReviewLease freeze is reached only after the upstream `review.claim` authorization succeeds.

26c. Binding suspension permits a valid callback for an already-issued award but prevents new delivery and new frozen work.

28.3 Contribution tests

27. Accept creates exactly one reviewer contribution.
28. Accept creates exactly one submitter contribution.
29. Needs revision creates exactly one reviewer contribution and no submitter contribution.
30. Reject creates exactly one reviewer contribution and no submitter contribution.
31. Review decision does not change reviewer award quantity.
32. Second completed revision review creates a second reviewer contribution.
33. Released lease creates no contribution.
34. Expired lease creates no contribution.
35. Revoked lease creates no contribution.
36. Automated checker result creates no Contribution Record.
37. Contribution project, task, assignment, submission, Review, and contributor chain must match.
38. Contribution Record cannot be updated.
39. Contribution Record cannot be deleted.
40. Global contributor view returns records from authorized projects.
41. Project filter returns only that project.

41a. Every ContributionRecord creates exactly one pending ContributionEvidenceProjection in the same transaction.

41b. Every ContributionRecord creates exactly one ContributionEvidenceProjectionRequested outbox event.

41c. Contribution source_submission_artifact_digest equals the judged SubmissionVersion digest.

41d. A completed-review contribution identifies the reviewed submission; it does not invent a reviewer-upload artifact.

28.4 Atomicity and idempotency tests

42. Failure creating reviewer contribution rolls back Review.
43. Failure creating submitter contribution rolls back accept Review.
44. Failure creating an award rolls back Review and all contributions.
45. Failure writing an outbox event rolls back Review, contributions, and awards.
46. External adapter outage does not roll back committed Review.
47. Exact decision replay returns existing Review, contributions, and awards.
48. Changed payload with reused decision idempotency key is rejected.
49. Concurrent decision attempts create one Review and one logical set of contributions.
50. Award uniqueness prevents duplicates under replay.

50a. Failure creating a pending evidence-projection row rolls back the Review transaction.

50b. Failure writing the evidence-projection outbox event rolls back the Review transaction.

50c. Artifact Storage outage after commit does not roll back Review, contributions or awards.

50d. Exact decision replay returns existing projection rows and creates no new projection event.

28.5 Compensation evaluation tests

51. Completed-review money award is copied exactly from frozen definition.
52. Completed-review points award is copied exactly from frozen definition.
53. Accepted-submission awards use TaskAssignment frozen version.
54. Reviewer awards use ReviewLease frozen version.
55. One contribution can create money and points awards independently.
56. Unpaid contribution creates zero awards.
57. Current project policy is never consulted instead of frozen reference.
58. Missing frozen version fails with integrity error and commits nothing.
59. Money award carries project_id and project binding.
60. Same points unit text in different projects remains separately scoped.

28.6 Delivery tests

61. CompensationFulfillmentRequested is committed with each award.
62. Dispatcher sends exact persisted payload.
63. Retry preserves event_id and idempotency_key.
64. Non-2xx adapter response leaves event unacknowledged.
65. 2xx acknowledgement marks delivery acknowledged but fulfillment pending.
66. Repeated delivery does not create another award.
67. Suspended binding pauses delivery without cancelling award.

67a. Finance Authority or operational Operator with `compensation.delivery.reconcile` may request eligible delivery reconciliation in covered scope.

67b. Project Manager, Contributor and foreign-project Admin are denied delivery reconciliation.

67c. Reconciliation reuses the existing event_id, payload, adapter binding and idempotency key and cannot mark acknowledgement or fulfillment.

28.7 Callback tests

68. Active bound service ActorProfile with `compensation.fulfillment.report` may report its own award.
69. Invalid Identity Issuer token is rejected.
70. Money adapter cannot report points award.
71. Project A adapter cannot report Project B award.
72. Unknown award is rejected.
73. Fulfilled callback requires external reference, quantity, and timestamp.
74. Fulfilled quantity must exactly equal award quantity.
75. Partial fulfillment is rejected.
76. Failed callback requires failure code.
77. Failed callback creates immutable receipt.
78. Failed followed by fulfilled is allowed.
79. Fulfilled followed by failed is rejected.
80. Exact callback replay returns existing receipt.
81. Reused external_event_id with different payload is rejected.
82. Concurrent fulfilled callbacks produce one fulfilled receipt.
83. Callback does not mutate Contribution Record.
84. Callback does not mutate CompensationAward.
85. Valid callback before delivery acknowledgement does not regress later.

85a. Human Admin, contributor, unprovisioned service actor, disabled actor and revoked identity link cannot report fulfillment.

85b. Valid service actor bound to another project or instrument is denied.

85c. Suspended binding may report an existing award it previously accepted, but cannot receive new delivery.

85d. Retired binding accepts only exact replay of an already accepted receipt.

85e. Accepted callback audit chain references the exact AuthorizationDecision.

85f. Callback request project, contributor, instrument, quantity or binding fields cannot grant or alter authority.

28.8 Read and recovery tests

86. Contribution detail composes award status without modifying canonical record.
87. Projection can be rebuilt from awards, delivery records, and receipts.
88. Reconciliation detects accepted Review missing submitter contribution.
89. Reconciliation detects Review missing reviewer contribution.
90. Reconciliation detects missing award.
91. Reconciliation detects unexpected award for unpaid policy.

92. Reconciliation detects missing fulfillment event.
93. Reconciliation detects projection/receipt mismatch.
94. Reconciliation never silently edits immutable records.
95. Binding retirement is rejected while an active policy, unfinished frozen work, or unfulfilled award depends on it.
96. Retiring the active policy without replacement clears the project active-version reference and blocks new work.
97. No cross-project Admin contributor-history or contributor-award endpoint exists in v0.1.
98. `contribution.read_self` returns only the caller's ContributionRecords and composed award summaries.
99. `contribution.read_project` filters rows, counts, totals and ages to covered canonical project scope.
100. `compensation.award.read` is required for administrative award detail.
101. Unauthorized contribution/evidence IDs do not reveal record existence.
102. Projection worker reloads canonical data rather than trusting the projection event payload.
103. Same contribution/schema version produces identical canonical bytes and digest under repeated generation.
104. Local Storage and Flow Node adapters pass the same evidence-projection conformance suite.
105. Verified provider receipt with wrong owner, project, source identity, artifact role, media type or digest is rejected.
106. Same artifact operation identity and bytes resolves to one verified ArtifactBinding.
107. Same artifact operation identity with changed bytes returns `artifact_idempotency_conflict` and binds nothing.
108. Contribution-list responses expose only evidence status and Workstream ArtifactBinding ID, not raw provider authority.
109. Authorized evidence retrieval verifies digest and creates an attributable access audit record.
110. Human Identity Issuer token is never forwarded to Artifact Storage or compensation adapters.
111. Artifact Storage outage returns the stable evidence error without changing contribution or compensation state.
112. Reconciliation detects missing projection row, missing event, invalid binding, digest mismatch and missing retention.
113. Authorized rebuild to a newer schema version retains the prior immutable bundle.
114. One live artifact-recovery attempt is enforced under concurrent Operator requests.
115. Same recovery request idempotency key returns the stored response with no new attempt, audit or job.
116. A different request while recovery is live returns `recovery_in_progress` with no mutation.
117. Worker crash reclaims the same recovery attempt with a higher execution generation; stale worker completion is fenced.
118. Recovery reuses the original provider idempotency identity and creates at most one logical artifact effect.
119. Unsafe ambiguity keeps the artifact operation dead-lettered and records one terminal audit result.
120. Operator requester identity never becomes an artifact-recovery execution lease owner.

120a. The same outbox/idempotency/recovery contract passes under Local Storage and Flow Node; recovery is not skipped merely because storage is local.

29. Implementation Delivery Order

Phase 0: Dependency contracts and authority catalogue

- confirm WS-AUTH-001 ActorProfile, AuthorizationDecision and canonical resource-context interfaces;
- consume the locked granular compensation permissions and `compensation.fulfillment.report` from WS-AUTH-001-A without aliases;
- reuse the shared `artifact.recovery.request` and `artifact.recovery.execute` permissions;
- extend the existing `artifact.binding.create` resource mapping to ContributionRecord for the fixed projection system actor and `contribution_evidence_bundle` role;
- confirm updated WS-REV-001 Review, ReviewFinding, FindingResolution, SubmissionVersion, ReviewLease and ArtifactBinding contracts;
- freeze the shared provider-independent ArtifactStorePort conformance contract;
- add contract tests that prevent local role-name checks, human-token forwarding and raw provider-reference disclosure.

Phase 1: Schema and immutable policy model

- CompensationPolicy and version tables;
- CompensationRule;
- award definitions;
- adapter bindings;
- frozen references on TaskAssignment and ReviewLease;
- ContributionRecord;
- CompensationAward;
- FulfillmentReceipt;
- status projection;
- ContributionEvidenceProjection;
- constraints and indexes.

Phase 2: Policy and binding APIs

- draft creation and validation;
- WS-AUTH permission/resource enforcement and immutable decision linkage;
- publish/activate/retire transaction;
- adapter binding lifecycle;
- explicit unpaid policy;
- concurrency guards.

Phase 3: Freeze integration

- TaskAssignment freeze;
- ReviewLease freeze;
- policy-change concurrency tests;
- legacy-data migration guard;
- suspended-binding behaviour.

Phase 4: Atomic Review integration

- reviewer contribution for all decisions;
- submitter contribution for accept;
- deterministic award evaluation;
- outbox events;
- pending evidence projections and evidence-projection outbox events;
- exact decision replay;
- rollback and race tests;
- removal of ContributionRecordRequested emission.

Phase 5: External adapter boundary

- outbox routing;
- delivery acknowledgement;
- callback authentication and authorization;
- exact bound service-actor permission evaluation;
- immutable receipts;
- status projection;
- conflict and idempotency handling.

Phase 6: Contribution evidence projection and authorized retrieval

- deterministic versioned evidence-bundle generator;
- ArtifactStorePort worker with Local Storage and Flow Node adapter conformance;
- verified generic ArtifactBinding creation and retention;
- authorized evidence retrieval and attributable access audit;
- retry, dead-letter, uncertain artifact recovery and projection reconciliation;
- no search-result disclosure before Workstream reauthorization.

Phase 7: Read APIs and operations

- global and project-filtered contribution views;
- compensation summaries;
- Admin operational views;

- metrics, tracing, alerts;
- reconciliation and projection rebuild.

Phase 8: Live API drill

The drill MUST prove at minimum:

1. paid accept producing reviewer and submitter contributions;
2. needs revision producing reviewer contribution only;
3. reject producing reviewer contribution only;
4. explicit unpaid policy;
5. policy change with frozen old and new work;
6. money and project-points awards from one contribution;
7. adapter acknowledgement without fulfillment;
8. failed callback followed by fulfilled callback;
9. exact decision and callback replay;
10. adapter outage with eventual delivery;
11. database evidence for atomicity and uniqueness;
12. projection rebuild and reconciliation.
13. Finance-only policy and binding mutation with Project Manager and Operator denials;
14. exact bound service-actor callback and human/foreign-adapter denials;
15. reviewer and submitter evidence-bundle projection through the configured ArtifactStorePort;
16. storage outage after contribution commit with canonical records unchanged;
17. authorized evidence retrieval with digest verification and attributable access;
18. uncertain artifact operation recovery with one live attempt and fenced worker generation;
19. Local Storage/Flow Node adapter conformance evidence where both adapters are available.

The drill report must clearly separate observed runtime proof from specification claims.

---

30. Definition of Done

WS-CON-001 is implemented only when:

- every valid Review creates exactly one reviewer Contribution Record;
- accept additionally creates exactly one submitter Contribution Record;
- no other outcome creates a submitter Contribution Record;
- review compensation is identical across accept, needs_revision, and reject under the same frozen policy;
- TaskAssignment and ReviewLease freeze immutable policy versions;
- policy replacement cannot alter existing work;
- explicit unpaid policy works without missing configuration;

- Contribution Records and CompensationAwards are immutable;
- money and project points remain project-scoped;
- Contribution Records remain globally addressable and project-filterable;
- Review, contribution, awards, and outbox events commit atomically;
- one pending ContributionEvidenceProjection and projection event commit atomically with every ContributionRecord;
- every protected human/service operation uses WS-AUTH-001 with the canonical resource context and immutable decision linkage;
- Finance Authority alone manages compensation policy and adapter bindings in v0.1;
- fulfillment callbacks require the exact bound active service actor and `compensation.fulfillment.report`;
- adapter outage cannot invalidate a committed Review;
- delivery acknowledgement and fulfillment are represented separately;
- callbacks are authenticated, authorized, idempotent, and exact-quantity;
- failed may become fulfilled, while fulfilled is terminal;
- fulfillment receipts are immutable;
- read projections are rebuildable;
- provider-specific payment and points internals are absent from Workstream;
- deterministic contribution-evidence bundles are stored only through the shared ArtifactStorePort;
- Local Storage and Flow Node satisfy the same Workstream evidence contract;
- public responses expose Workstream ArtifactBinding references, never raw provider authority;
- authorized evidence reads verify integrity, recheck scope and create attributable access records;
- storage failure and uncertain recovery cannot rewrite canonical Review, contribution or compensation facts;
- human tokens are never forwarded to Artifact Storage or compensation adapters;
- all conformance tests pass;
- the live drill captures authorization, API, database, event, artifact, retry, recovery, callback, and audit evidence.

31. Explicitly Out of Scope

The coding agent MUST NOT implement any of the following under WS-CON-001:

- payment-request objects;
- payment-attempt objects;
- payment-provider SDK logic;
- bank, wallet, card, stablecoin, x402, or settlement execution;
- payout batching;
- balances or account statements;
- KYC or beneficiary onboarding;

- finance-approval workflow inside Workstream;
- provider retry or provider reconciliation logic;
- project-points ledger;
- point balances or transfers;
- Workstream-wide points;
- credits or credit ledger;
- token issuance;
- compensation tiers;
- task-specific compensation overrides;
- contributor-specific rates;
- skill- or reputation-derived rates;
- bonuses, penalties, multipliers, or dynamic formulas;
- partial fulfillment;
- currency conversion;
- reputation score calculation;
- reputation aggregate maintenance;
- automated reviewer-role grants;
- adjudication or second-level review;
- contribution reversal, voiding, or adjustment;
- retroactive policy application;
- mutation or deletion of canonical Contribution Records, awards, or receipts;
- a second contribution-specific artifact store;
- direct Local Storage, Flow Node, filesystem or CID calls outside ArtifactStorePort;
- treating Artifact Storage as the canonical source for a ContributionRecord, Review, award or fulfillment result;
- public raw provider URLs, CIDs, filesystem paths or provider capability tokens;
- forwarding human Identity Issuer tokens to Artifact Storage or compensation adapters;
- using semantic search results as disclosure authority without Workstream reauthorization;
- changing canonical contribution history because evidence projection, indexing or retention failed;
- Operator ownership or cross-Operator takeover of artifact-recovery execution leases;
- unrelated product architectures.

External adapters may later connect to those systems, but this specification defines only the Workstream-facing contract.

32. Future Additive Directions

The following may be designed later through separate specifications:

- provider-specific money adapters;

- project-points service and ledger;
- credits and other compensation instruments;
- finance approval adapters;
- task-category and compensation-tier policy rules;
- bonuses or quality incentives based on independently verified evidence;
- partial fulfillment;
- compensation adjustment and reversal records;
- adjudication and audit review;
- Workstream-wide economic reporting;
- reputation scoring and project-to-global aggregation;
- automated ProjectRoleGrant policy;
- external evidence publication.

Future additions MUST preserve:

- original Contribution Records;
- original CompensationAwards;
- frozen policy references;
- original fulfillment receipts;
- decision-neutral base reviewer compensation;
- project scope;
- event idempotency.

33. Coding-Agent Handoff Rules

The coding agent MUST follow these rules without reinterpretation:

1. Implement this specification as an extension of WS-REV-001, not as a separate competing lifecycle.
2. Replace the temporary ContributionRecordRequested seam with direct atomic creation.
3. Keep the combined Review transaction short and database-local.
4. Do not call external adapters inside the Review transaction.
5. Do not use current project policy when a frozen assignment or lease version exists.
6. Do not derive reviewer compensation from Review.decision.
7. Do not create submitter contribution for needs_revision or reject.
8. Do not create contribution for an incomplete lease.
9. Do not store fulfillment state on ContributionRecord or CompensationAward.
10. Do not build payment, point-ledger, credit, or provider domain models.
11. Use exact decimal types and decimal-string API fields.
12. Enforce uniqueness in PostgreSQL, not only application code.
13. Require idempotency for Review decisions, event consumers, adapter delivery, and callbacks.

14. Treat external adapter payloads as untrusted.
15. Preserve project scope on every economic record and event.
16. Keep global contribution views as queries/read models, not a second global ledger.
17. Use append-only receipts and rebuildable projections.
18. Never silently repair immutable history.
19. Add no adjudication, appeal, reversal, or adjustment path.
20. Add no conditional compensation rule beyond exact contribution_type matching.
21. If existing code conflicts with a MUST rule, stop and raise the conflict before weakening the specification.
22. Completion requires conformance tests and a live API drill report; code compilation alone is insufficient.
23. Use WS-AUTH-001 AuthorizationService for every protected action; do not add local role-name checks or a second permission system.
24. Finance Authority alone manages compensation policy and adapter bindings in v0.1.
25. Require `compensation.fulfillment.report` from the exact frozen adapter service actor; no human role can satisfy a callback.
26. Load canonical resource ownership before authorization and revalidate sensitive writes inside the transaction.
27. Create one pending ContributionEvidenceProjection and event with every ContributionRecord.
28. Generate evidence asynchronously from canonical committed records; never trust the projection event as independent truth.
29. Use the shared ArtifactStorePort and generic ArtifactBinding contract; do not import provider internals or create another byte store.
30. Never forward human tokens to Artifact Storage or compensation adapters.
31. Never expose raw provider CIDs, paths, URLs, credentials or search candidates as authority.
32. Treat Local Storage and Flow Node as interchangeable adapters at the Workstream contract boundary and run the same conformance suite against both.
33. Keep artifact projection/retrieval failure separate from contribution and compensation validity.
34. For uncertain artifact operations, reuse the original idempotency identity, permit one live recovery attempt and let only a fenced system worker own execution.
35. Link policy, binding, callback, read, recovery and projection operations to immutable AuthorizationDecision and audit records without storing bearer tokens.
36. Consume the exact granular identifiers in WS-AUTH-001-A. Do not implement removed broad aliases such as `compensation.policy.manage`, `compensation.adapter_binding.manage`, `operations.reconcile.run`, `operations.outbox.retry`, or `review.decision`.

34. Final Implementation Contract

The complete v0.1 contract is:

> Workstream records every valid completed Review as one immutable reviewer contribution, regardless of decision. An accepted Review additionally records one immutable submitter contribution. Each contribution is evaluated against compensation terms frozen before work began. Workstream creates immutable, project-scoped money or project-points awards and emits transactional fulfillment instructions. Exact bound service actors perform provider and ledger work and report immutable fulfilled or failed results. In the same canonical transaction, Workstream schedules one deterministic evidence-bundle projection per contribution; after commit, the shared ArtifactStorePort retains and verifies the bundle under a Workstream ArtifactBinding. WS-AUTH-001 governs every human and service action. Contribution and award history never changes; API views compose fulfillment and evidence status from append-only facts and rebuildable projections without exposing raw provider authority.